



FACULTAD DE INGENIERIA

Universidad de Buenos Aires

Captación del entorno y mapeo local

Informe de tesis de Ingeniería Electrónica

Autor	Tomás de Aguirre	103887	tdeaguirre@fi.uba.ar
Tutor	Ignacio Mas	206872	imas@fi.uba.ar
Co-tutor	Sergio Lew	128563	slew@fi.uba.ar

Resumen

Las personas con ceguera o baja visión enfrentan dificultades significativas al desplazarse por entornos desconocidos o dinámicos. Las herramientas de asistencia tradicionales, como el bastón blanco, presentan limitaciones en alcance y resolución espacial, y no advierten eficazmente obstáculos ubicados a la altura de la cabeza. En paralelo, disciplinas como la robótica móvil han consolidado un conjunto de tecnologías de percepción, mapeo y comunicación entre procesos que permiten a vehículos autónomos interpretar su entorno en tiempo real, pero que rara vez se aplican al problema de la movilidad personal asistida.

El eje central de este trabajo, en tanto tesis de ingeniería, es estudiar, evaluar y adaptar esas tecnologías, entre ellas la percepción de profundidad por visión estéreo activa, las grillas de ocupación probabilísticas, la fusión de datos inerciales y el ecosistema ROS 2, para determinar las alternativas más adecuadas con las que construir una herramienta de asistencia funcional, portátil y de bajo costo. El aporte no reside en la creación de técnicas nuevas, sino en la selección, integración y validación de soluciones ya probadas en otros rubros, evaluando sus compromisos de diseño bajo las restricciones propias de un dispositivo de asistencia personal: cómputo embebido limitado, portabilidad y baja latencia. La elección del canal háptico como interfaz de salida se fundamenta en el principio de sustitución sensorial: el cerebro puede reorganizarse para interpretar estímulos táctiles como información espacial cuando la visión está reducida o ausente. Sobre esa base, el proyecto global busca traducir información espacial visual en estímulos hápticos interpretables para la navegación; se estructura en dos etapas complementarias, (i) captación del entorno y mapeo local y (ii) traducción a interfaz háptica, y la presente tesis, enmarcada en la modalidad *Tesis de Ingeniería Electrónica*, aborda exclusivamente la primera.

En concreto, se desarrolla en ROS 2 un módulo de percepción que, a partir de la cámara Intel RealSense D435i montada sobre la cabeza del usuario y ejecutándose en una Raspberry Pi 5, genera en tiempo real una representación compacta del entorno inmediato. El desarrollo sigue tres versiones incrementales. La v0 resuelve el problema de forma directa y con baja latencia: reduce la imagen de profundidad a una grilla de 5 filas $\times$ 10 columnas, la convierte en intensidades hápticas, las visualiza mediante `output_viewer` y genera la trama UART prevista para una futura interfaz física. Sobre esa base, la v1 mejora la interpretación de las mediciones y la señal destinada a la segunda etapa: alinea la nube con la gravedad, elimina piso y techo, y mejora la distribución espacial asociada a cada motor. Por último, la v2 amplía la protección fuera del campo de visión: usa la salida filtrada de v1 para construir un mapa local de ocupación 2D egocéntrico basado en log-odds, con trazado de rayos Bresenham, desplazamiento dinámico (*scroll*) y odometría por RGB-D e IMU, conservando información reciente de obstáculos que han dejado de estar dentro del campo de visión de la cámara. Un nodo de fusión combina esta memoria espacial con la percepción frontal instantánea de v1.

El desempeño se evalúa mediante métricas cuantitativas de error de profundidad, latencia extremo a extremo, precisión y *recall* en detección de obstáculos, estabilidad temporal del mapa local y consumo de recursos computacionales en la plataforma embebida objetivo.

Abstract

People who are blind or have low vision face significant difficulties when navigating unfamiliar or dynamic environments. Traditional assistive tools such as the white cane have limited range and spatial resolution, and fail to effectively detect obstacles at head height. Meanwhile, fields such as mobile robotics have consolidated a set of perception, mapping and inter-process communication technologies that enable autonomous vehicles to interpret their surroundings in real time, yet these technologies are seldom applied to the problem of personal assisted mobility.

The core focus of this work, as an engineering thesis, is to study, evaluate and adapt those technologies, including active stereo depth sensing, probabilistic occupancy grids, inertial data fusion and the ROS 2 ecosystem, in order to identify the most suitable alternatives for building a functional, portable and low-cost assistive device. The contribution does not lie in the creation of novel techniques but rather in the selection, integration and validation of solutions already proven in other domains, assessing their design trade-offs under the constraints inherent to a personal assistive device: limited embedded computing, portability and low latency. The choice of the haptic channel as the output interface is grounded in the principle of sensory substitution: the brain can reorganize to interpret tactile stimuli as spatial information when vision is reduced or absent. On that basis, the broader project seeks to translate visual spatial information into haptic stimuli interpretable for navigation; it is structured in two complementary stages: (i) environment capture and local mapping, and (ii) translation into a haptic interface. This thesis, submitted under the *Electronic Engineering Thesis* track, addresses exclusively the first stage.

Specifically, a perception module is developed in ROS 2 that, using an Intel RealSense D435i camera mounted on the user's head and running on a Raspberry Pi 5, generates a compact real-time representation of the immediate surroundings. The development follows three incremental versions. The v0 version addresses the problem directly and with low latency: it reduces the depth image to a grid of 5 rows by 10 columns, converts it to haptic intensities, displays them through `output_viewer`, and generates the UART frame intended for a future physical interface. Building on this basis, v1 improves measurement interpretation and the signal intended for the second stage: it aligns the point cloud with gravity, removes floor and ceiling points, and improves the spatial distribution assigned to each motor. Finally, v2 extends protection beyond the field of view by using the filtered output of v1 to build and maintain a 2D egocentric local occupancy map based on log-odds, with Bresenham ray casting, dynamic scrolling and RGB-D and IMU odometry, preserving recent obstacle information even after obstacles leave the camera's field of view. A fusion node combines this spatial memory with the instantaneous frontal perception of v1.

System performance is assessed through quantitative metrics including depth error, end-to-end latency, obstacle detection precision and recall, temporal stability of the local map, and computational resource consumption on the target embedded platform.

Dedicatoria

Agradecimientos

Índice general

1. Introducción	11
1.1. Contexto y motivación	11
1.2. Planteo del problema	11
1.3. Objetivos	12
1.3.1. Objetivo del proyecto	13
1.3.2. Objetivo de la tesis	13
1.3.3. Objetivo técnico	13
1.4. Contribuciones del trabajo	14
1.5. Alcance	14
1.6. Organización del documento	15
2. Estado del arte	16
2.1. Dispositivos de asistencia sensorial para personas ciegas	16
2.1.1. Criterios de adopción	16
2.1.2. Sistemas sin RGB-D (sensado puntual)	17
2.1.3. Sistemas con RGB-D (percepción densa)	17
2.1.4. Producto comercial consolidado: <i>SuperBrain 1</i>	18
2.1.5. Síntesis comparativa de los sistemas revisados	19
2.2. Mapeo local y evitación de obstáculos en robótica móvil	20
2.2.1. Mapas locales reactivos	20
2.2.2. Representaciones de ocupación probabilística	20
2.2.3. Representación 2D y compensación gravitacional	21
2.2.4. Estimación del movimiento egocéntrico	21
2.3. Brecha identificada y posicionamiento de la tesis	21
3. Marco teórico	24
3.1. Fundamentos de la versión v0: percepción directa, ROS 2 y salida háptica	24
3.1.1. Sustitución sensorial y neuroplasticidad	24
3.1.2. Principio físico de la cámara Intel RealSense D435i	25
3.1.3. Modelo de cámara <i>pinhole</i> y proyección inversa	26
3.1.4. Limitaciones por superficies especulares e iluminación intensa	27
3.1.5. ROS 2 como middleware de integración	27
3.1.6. Reducción espacial de la imagen de profundidad	28
3.1.7. Codificación angular de la nube filtrada	29
3.1.8. Mapeo de distancia a comando de intensidad	30
3.1.9. Comunicación serie embebida	31
3.2. Fundamentos de la versión v1: orientación, gravedad y suelo	32
3.2.1. Ángulos de cámara y convención óptica	32
3.2.2. Singularidad de Euler y cuaterniones	33

3.2.3.	Alineación gravitacional con acelerómetro	34
3.2.4.	Filtrado inercial local para la dirección de gravedad	35
3.2.5.	Integración de giróscopo y deriva de yaw	35
3.2.6.	Detección del plano del suelo	36
3.2.7.	Estimación adaptativa de ruido	36
3.2.8.	Submuestreo por voxel grid	37
3.2.9.	RANSAC y refinamiento por mínimos cuadrados	37
3.2.10.	Robustez ante suelo parcialmente visible	38
3.2.11.	Clasificación geométrica y región superior excluida	39
3.2.12.	Realimentación de la altura de cámara	40
3.3.	Fundamentos de la versión v2: odometría, mapa local y conciencia espacial	40
3.3.1.	Filtro de Madgwick para orientación IMU	40
3.3.2.	Odometría RGB-D con RTAB-Map	42
3.3.3.	Marcos de referencia y convención REP-103	42
3.3.4.	Sincronización temporal de observaciones y odometría	43
3.3.5.	Representaciones espaciales locales	43
3.3.6.	Grillas de ocupación y log-odds	43
3.3.7.	Ray casting con Bresenham	45
3.3.8.	Ventana local deslizante y olvido espacial	46
3.3.9.	Filtrado espacial y temporal	46
3.3.10.	Conciencia espacial y tiempo hasta colisión	46
3.3.11.	Fusión háptica con riesgo contextual	48
3.3.12.	Métricas de evaluación para percepción y mapeo	49
4.	Arquitectura del sistema	50
4.1.	Arquitectura general	50
4.2.	Evolución arquitectónica por versiones (v0, v1, v2)	50
4.3.	Paquetes de soporte e infraestructura	52
4.4.	Adquisición y normalización de tópicos	52
4.5.	Versión v0: solución frontal directa	53
4.6.	Versión v1: refinamiento de la percepción frontal	53
4.7.	Versión v2: memoria espacial y protección contextual	54
4.8.	Conciencia espacial	55
4.9.	Fusión háptica	55
4.10.	Interfaz lógica de salida y transmisión UART	55
4.11.	Visualización y depuración	56
5.	Diseño del método (pipeline de percepción)	57
6.	Implementación en ROS 2	58
7.	Metodología experimental	59
8.	Resultados	60
9.	Discusión	61
10.	Conclusiones	62
11.	Trabajo futuro	63

Anexos

64

Índice de figuras

1.1. Visión general del sistema de asistencia sensorial desarrollado en esta tesis. El sistema que corre en la Raspberry Pi 5 genera una representación compacta del entorno destinada a una futura interfaz háptica portada por el usuario.	12
3.1. Geometría estéreo rectificada. Un mismo punto se proyecta en las coordenadas horizontales u_L y u_R de los planos virtuales de imagen; su diferencia define la disparidad d	26
3.2. Reducción espacial de una imagen de profundidad de 1280×720 píxeles a una DepthGrid de 5 filas por 10 columnas. La captura proviene de la ejecución de <code>v0</code>	29
3.3. Imagen de profundidad de entrada y codificación angular y vertical de su nube de obstáculos filtrada en una DepthGrid de 5 filas por 10 columnas. La reconstrucción fuera de línea coincide celda por celda con las estadísticas publicadas por <code>obstacle_grid_encoder</code>	30
3.4. Conversión de la distancia mínima válida en intensidad lógica. Las mediciones válidas entre 3 y 5 m permanecen en DepthGrid , pero producen intensidad nula; una celda sin mediciones válidas contiene estadísticas <code>NaN</code> y queda inactiva.	31
3.5. Ángulos de Euler en la convención óptica de la cámara. Cada rotación se representa alrededor de un eje fijo del sensor.	33
3.6. Singularidad <i>gimbal lock</i> : con $\text{pitch} = -90^\circ$ la cámara apunta verticalmente hacia abajo y los ejes de roll y yaw quedan alineados.	33
3.7. Ventajas de la representación mediante cuaterniones para orientaciones compuestas y configuraciones singulares.	34
3.8. Cámara D435i en posición horizontal. El acelerómetro mide $(0, -g, 0)$ en la convención óptica usada por el pipeline.	35
3.9. Casos límite en el cálculo de la rotación mínima entre el vector de aceleración y la vertical canónica.	36
3.10. Esquema tridimensional de submuestreo por <i>voxel grid</i> . Los puntos originales dentro de cada celda ocupada se reemplazan por un único representante ubicado en el centroide de las muestras. Elaboración propia.	38
3.11. Esquema bidimensional de RANSAC para ajuste de planos. Cada hipótesis se genera a partir de tres puntos y luego se cuentan como <i>inliers</i> las muestras cuya distancia al plano candidato es menor que la tolerancia ε . Elaboración propia.	39
3.12. Diagrama de bloques del filtro de Madgwick para una implementación IMU sin magnetómetro. La estimación propagada mediante el giróscopo se corrige por descenso de gradiente a partir de la dirección de gravedad observada por el acelerómetro, y luego se integra y normaliza. Fuente: Madgwick [32, Fig. 2].	41
3.13. Evolución del log-odds de una celda inicialmente desconocida y conversión de cada estado al valor publicado en <code>OccupancyGrid.data[i]</code> , utilizando la configuración inicial previa a la calibración.	45

3.14. Recorrido discreto de Bresenham sobre la grilla: las celdas intermedias reciben evidencia libre; ante un obstáculo el extremo es ocupado y, en caso contrario, la evidencia libre se extiende hasta el límite del rango representado.	46
3.15. Geometría empleada para seleccionar un riesgo contextual y calcular el tiempo hasta colisión. La zona frontal observable queda a cargo de la percepción frontal; la evaluación contextual considera las regiones laterales y posterior.	48
4.1. Arquitectura v0.	51
4.2. Arquitectura v1.	51
4.3. Arquitectura v2.	51
4.4. Mensajes funcionales de v0. La imagen de profundidad se reduce a DepthGrid , se transforma en una matriz lógica HapticGrid y se entrega a la interfaz de transmisión.	53
4.5. Mensajes funcionales de v1. La imagen, los intrínsecos y el acelerómetro permiten obtener una nube de obstáculos alineada y codificarla conservando el contrato DepthGrid-HapticGrid de v0.	54
4.6. Mensajes funcionales de v2. La ruta frontal se complementa con odometría, memoria local y riesgo contextual, que se fusiona antes de publicar la matriz háptica final. Se muestran únicamente los mensajes que intervienen en el procesamiento funcional principal.	54

Índice de tablas

2.1.	Comparación de sistemas de asistencia sensorial para personas ciegas según los criterios de Dakopoulos y Bourbakis [13] y dos criterios técnicos adicionales. ✓: satisfecho; ~: parcialmente satisfecho; ×: no satisfecho. La fila «Proyecto global» considera la integración de las dos etapas del proyecto.	20
2.2.	Alternativas consideradas para estimar el movimiento del mapa local y configuración adoptada con <code>rtabmap_odom</code>	22
4.1.	Correspondencias de tópicos implementadas mediante remapeos en <code>nav.launch.py</code> . En esta tabla, <code>RS/</code> abrevia el prefijo <code>/camera/camera/</code>	52
1.	Trazabilidad de adquisición, prefiltrado IMU y percepción geométrica.	65
2.	Trazabilidad de mapeo local, codificación de grilla y conciencia espacial.	65
3.	Trazabilidad de fusión háptica, salida, visualización y orquestación.	65

Capítulo 1

Introducción

1.1. Contexto y motivación

Las personas con ceguera o baja visión enfrentan dificultades significativas al desplazarse por entornos desconocidos o dinámicos. Las herramientas de asistencia tradicionales resuelven funciones diferentes. El bastón blanco aporta información por contacto sobre el espacio inmediato y el estado del suelo, pero su alcance queda limitado por la longitud y el barrido realizado por el usuario. El perro guía puede anticipar y evitar obstáculos durante la marcha, aunque no comunica una representación espacial explícita del entorno. En ambos casos, una interfaz electrónica puede aportar información complementaria sobre objetos elevados, como ramas, cables o señales colgantes, antes de que entren en contacto con el usuario.

En los últimos años han surgido dispositivos electrónicos de asistencia que incorporan sensores activos [1]-[3]. Sin embargo, gran parte de esas soluciones se basa en sensores puntuales de corto alcance, lo que restringe la cantidad y calidad de información espacial disponible para el usuario. En contraste, los sensores de profundidad por visión estéreo activa, como la familia Intel RealSense [4], permiten obtener una medición densa con información tridimensional dentro de su campo de visión. Su adecuación a una plataforma portátil y de baja latencia debe evaluarse junto con el procesamiento ejecutado sobre el hardware embebido.

El presente trabajo se enmarca en el principio de neuroplasticidad sensorial, según el cual el cerebro puede reorganizarse para explotar de manera más eficiente los sentidos restantes cuando la visión está reducida o ausente [5]. Sobre esa base, el proyecto global del que forma parte esta tesis busca diseñar un sistema portátil capaz de traducir información espacial visual en estímulos hápticos interpretables para la navegación asistida.

Si bien los trabajos académicos que combinan cámaras RGB-D con retroalimentación háptica [6], [7] demuestran la viabilidad técnica del enfoque, y desarrollos comerciales como el *SuperBrain 1* [8] constituyen un referente de lo que es posible alcanzar en términos de funcionalidad y validación real, el costo y la escala de desarrollo de esas soluciones las sitúan fuera del alcance de la mayoría de los usuarios. Esta tesis estudia cómo obtener una representación local compacta de obstáculos mediante una cámara RGB-D y una Raspberry Pi 5, sin pretender reproducir el alcance funcional ni el nivel de validación de un producto comercial. El Capítulo 2 analiza estas soluciones en detalle y posiciona la contribución de esta tesis respecto de ellas.

1.2. Planteo del problema

Una persona con ceguera o baja visión que se desplaza por un entorno desconocido puede beneficiarse de información actualizada sobre obstáculos ubicados desde el suelo hasta la altura

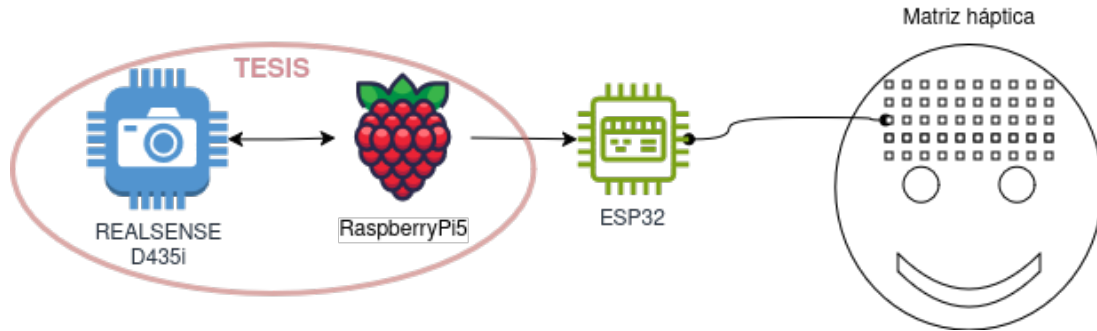


Figura 1.1: Visión general del sistema de asistencia sensorial desarrollado en esta tesis. El sistema que corre en la Raspberry Pi 5 genera una representación compacta del entorno destinada a una futura interfaz háptica portada por el usuario.

de la cabeza. El bastón blanco, el perro guía y los dispositivos electrónicos ofrecen capacidades complementarias, pero no comunican el entorno con la misma cobertura, resolución ni modalidad. Los dispositivos basados en ultrasonido o sensores puntuales ofrecen información espacial acotada [2], [3]; los sistemas con cámaras de profundidad aportan mediciones más densas, a costa de una mayor demanda de procesamiento [6], [7].

El problema de ingeniería que aborda esta tesis es, por lo tanto, transformar información tridimensional del entorno en una representación compacta destinada a una interfaz háptica, utilizando una plataforma embebida y manteniendo acotados el retardo, la variación temporal y el consumo de recursos.

Las restricciones determinantes del diseño son:

- **Portabilidad:** se registrarán la masa, las dimensiones y la potencia eléctrica del conjunto implementado formado por cámara y unidad de cómputo. [FALTA DEFINIR los valores máximos admisibles y medir el conjunto final].
- **Cómputo embebido limitado:** el sistema se ejecutará en una Raspberry Pi 5 [9], sin depender de CUDA ni de una GPU discreta. Se medirán uso de CPU, memoria, temperatura y eventos de limitación térmica. [FALTA DEFINIR los límites de aceptación y completar las mediciones].
- **Latencia:** se medirá el retardo extremo a extremo desde la adquisición hasta la publicación de la salida lógica. [FALTA DEFINIR la latencia máxima admisible y medir mediana, percentil 95 y peor caso].
- **Estabilidad temporal:** se cuantificarán la variación de la salida ante una escena estática, la tasa de cuadros sin detección y el tiempo durante el cual persiste una ocupación después de desaparecer la evidencia. [FALTA DEFINIR los umbrales de aceptación y realizar las mediciones].

1.3. Objetivos

Los objetivos del trabajo se articulan en tres niveles complementarios: el objetivo global del proyecto en el que se inscribe esta tesis, el objetivo específico de la tesis dentro de ese proyecto, y el objetivo técnico que define con precisión los requerimientos del sistema a implementar.

1.3.1. Objetivo del proyecto

El objetivo final del proyecto es desarrollar un sistema portátil de asistencia sensorial que permita a personas con ceguera o baja visión percibir su entorno mediante estímulos táctiles, mejorando su seguridad, autonomía y capacidad de desplazamiento independiente. El proyecto se estructura en dos etapas complementarias: (i) captación del entorno y mapeo local (objeto de esta tesis) y (ii) implementación física y validación de la interfaz háptica, desarrolladas en un trabajo separado. Como interfaz entre ambas etapas, esta tesis genera los comandos de intensidad, permite visualizarlos y deja implementada su transmisión mediante UART.

1.3.2. Objetivo de la tesis

Esta tesis aborda exclusivamente la primera etapa: captar el entorno y generar una representación local compacta a partir de percepción RGB-D. El foco se sitúa en transformar la medición cruda de profundidad en información espacial destinada a navegación asistida y caracterizar su retardo, estabilidad temporal y consumo de recursos sobre una plataforma embebida.

1.3.3. Objetivo técnico

El objetivo técnico es desarrollar un módulo que transforme mediciones de profundidad en una representación espacial compacta y en una matriz lógica de intensidades destinada a la segunda etapa del proyecto. Para materializar y evaluar ese objetivo se adoptan ROS 2 [10], una cámara Intel RealSense D435i [4] y una Raspberry Pi 5 como decisiones de implementación.

Específicamente, el módulo deberá desarrollarse en tres niveles incrementales:

- **v0 (percepción instantánea):** procesar imágenes de profundidad mediante reducción de dimensionalidad para producir una grilla estructurada de baja resolución con estadísticas por celda (distancia mínima, media y conteo de puntos válidos), y publicar dicha salida en tópicos ROS 2 estandarizados, visualizarla sin depender del hardware físico y serializarla para su transmisión mediante UART. Esta versión establece la referencia funcional directa con la cual comparar el costo de las etapas posteriores;
- **v1 (percepción filtrada):** incorporar etapas de filtrado de la medición para mejorar la relevancia de la señal destinada a navegación asistida, suprimiendo contribuciones espurias del entorno y estabilizando la representación ante variaciones de pose del portador. Su efecto sobre las activaciones falsas y la consistencia de la salida se evaluará experimentalmente;
- **v2 (mapeo local):** implementar una capa de mapeo de corto alcance basada en grillas de ocupación log-odds [11], [12], con memoria temporal de obstáculos, mecanismo de desplazamiento de ventana egocéntrica e integración de información inercial para compensar cambios de orientación durante la actualización del mapa. Su capacidad para conservar información sobre obstáculos fuera del campo frontal se evaluará en los escenarios definidos para v2.

Los parámetros numéricos y umbrales de decisión se refinarán mediante una fase de calibración y pruebas experimentales, descritas en el Capítulo 7.

1.4. Contribuciones del trabajo

Los aportes técnicos cuya validez se analiza mediante la evaluación experimental son los siguientes:

- una arquitectura incremental que permite comparar una transformación directa de profundidad con una interpretación geométrica filtrada y con una extensión dotada de memoria espacial, manteniendo una interfaz lógica común;
- una estrategia de filtrado geométrico alineada con la gravedad para estudiar la reducción de activaciones producidas por piso y techo sobre cómputo embebido;
- una representación compacta que separa la percepción métrica frontal del riesgo contextual y permite verificar ambas señales sin depender de la interfaz háptica física;
- un esquema de memoria local y fusión destinado a estudiar la continuidad de advertencia cuando un obstáculo previamente observado sale del campo de visión;
- un protocolo experimental reproducible para caracterizar error, detección, latencia, estabilidad temporal y consumo de recursos.

1.5. Alcance

El alcance de esta tesis comprende:

- el diseño, implementación y validación experimental de la etapa de captación del entorno del sistema de asistencia sensorial;
- la integración del módulo desarrollado con el ecosistema ROS 2, la generación y visualización de la matriz de intensidades y su serialización para transmisión mediante UART;
- la generación de representaciones compactas de un mapa de ocupación para navegación asistida sobre hardware embebido;
- la caracterización cuantitativa del desempeño del sistema en términos de precisión, latencia y consumo de recursos.

Esta tesis incluye la generación de la matriz de intensidades hápticas, su publicación mediante mensajes ROS 2, su visualización mediante una interfaz virtual y la implementación del enlace UART previsto para integrarla con el hardware de actuación. Quedan *fuera* de su alcance el diseño y la implementación de la interfaz háptica física —microcontrolador receptor, electrónica de potencia, actuadores y soporte corporal—, así como su caracterización y la validación con usuarios finales.

La plataforma embebida adoptada es una Raspberry Pi 5 [9]. Las pruebas experimentales caracterizarán el uso de CPU, memoria y latencia para determinar el margen operativo de la implementación sobre esa plataforma, no para decidir posteriormente su selección.

1.6. Organización del documento

El presente documento se organiza de la siguiente manera. La evolución v0–v1–v2 definida en la Sección 1.3.3 funciona como hilo conductor; cada capítulo la aborda desde una perspectiva diferente para evitar repetir su descripción.

El Capítulo 2 revisa el estado del arte en dispositivos de asistencia sensorial con retroalimentación háptica y en técnicas de mapeo local y evitación de obstáculos en robótica móvil, posicionando la contribución de esta tesis respecto de las soluciones existentes. El Capítulo 3 desarrolla los fundamentos teóricos del sistema: percepción de profundidad por estereoscopia activa, modelo de cámara *pinhole*, reducción espacial mediante partición en regiones de interés, grillas de ocupación log-odds, algoritmo de Bresenham, estimación de orientación con IMU y protocolo de comunicación serie. El Capítulo 4 describe la arquitectura modular del sistema implementada en ROS 2 y los cambios de topología introducidos en cada versión. El Capítulo 5 justifica las decisiones de procesamiento y los criterios mediante los cuales las mediciones afectan la salida. El Capítulo 6 expone la implementación concreta en C++ y Python sobre ROS 2, describiendo los nodos principales (`depth_to_matrix_node`, `local_mapper_node`, `haptic_grid_generator`, `hardware_manager`) y las interfaces definidas. El Capítulo 7 presenta la metodología experimental, los escenarios de prueba y las métricas de evaluación utilizadas: error RMS de profundidad, latencia *end-to-end*, precisión y *recall* de detección de obstáculos, estabilidad temporal por celda y consumo de recursos. El Capítulo 8 expone los resultados empíricos obtenidos y su análisis cuantitativo. Finalmente, los Capítulos 9, 10 y 11 discuten las limitaciones del enfoque, formulan las conclusiones del trabajo y proponen líneas de trabajo futuro, respectivamente.

Capítulo 2

Estado del arte

2.1. Dispositivos de asistencia sensorial para personas ciegas

Este capítulo presenta una revisión narrativa de antecedentes seleccionados por su relación con las decisiones centrales de la tesis: modalidad de sensado, forma de realimentación, resolución espacial, memoria temporal y plataforma de cómputo. La selección no pretende ser exhaustiva ni sostener afirmaciones sobre toda la bibliografía disponible; las conclusiones se limitan a los trabajos incluidos y citados en las secciones siguientes.

El principio de sustitución sensorial, cuya base experimental se describe en el Capítulo 3 (Section 3.1.1), constituye el fundamento común de los dispositivos electrónicos de asistencia a la movilidad revisados en este capítulo. Desde los primeros arreglos de 20×20 taxeles (píxeles táctiles) de Bach-y-Rita [5], la evolución técnica puede leerse como una continuidad: se parte de soluciones con sensado puntual y se avanza hacia sistemas con percepción densa mediante cámaras de profundidad, en búsqueda de mejor resolución espacial sin perder portabilidad ni baja latencia.

A partir del trabajo de Bach-y-Rita, se desarrolló una amplia variedad de dispositivos electrónicos de ayuda a la movilidad (*Electronic Travel Aids*, ETA). Dakopoulos y Bourbakis [13] presentan una revisión exhaustiva de estos sistemas, clasificándolos según el tipo de sensor (ultrasónico, infrarrojo, láser, cámara), el mecanismo de retroalimentación (auditivo, háptico, voz) y la zona del cuerpo en la que presentan la estimulación (mano, muñeca, torso, cabeza). Sobre esa base, la revisión se organiza como una progresión tecnológica dentro de un mismo problema de asistencia sensorial, estructurada en dos grupos: sistemas sin RGB-D, basados en sensado puntual, y sistemas con RGB-D, que incorporan percepción densa y, en algunos casos, mapeo local.

2.1.1. Criterios de adopción

Para encuadrar la evaluación comparativa, Dakopoulos y Bourbakis enumeran cuatro criterios que un dispositivo portátil de asistencia debe cumplir para resultar adoptable en la práctica: (1) *manos libres*, sin requerir el uso de las manos para un dispositivo *adicional* al bastón blanco, que se asume como herramienta base y cuyo uso no viola este criterio; (2) *oídos libres*, sin interferir con la capacidad de escucha del entorno, que es una fuente de información crítica; (3) *vestible*, para ofrecer flexibilidad y aprovechar las ventajas de las tecnologías portátiles; y (4) *simple*, de operación intuitiva y sin necesidad de entrenamiento extenso. A estos criterios se agregan dos dimensiones técnicas adicionales relevantes para el presente trabajo: *resolución*

espacial del sensor y *memoria temporal* en la representación del entorno.

2.1.2. Sistemas sin RGB-D (sensado puntual)

Entre las propuestas sin uso de RGB-D incluidas en esta revisión se distinguen tres enfoques:

Arreglos vibrotáctiles corporales

Novich y Eagleman [1] estudiaron los límites perceptuales de la piel como canal de información y concluyeron que un arreglo de actuadores sobre el torso puede alcanzar tasas de transferencia del orden de bits por segundo, suficientes para codificar patrones espaciales rudimentarios. Desde el punto de vista de los criterios de adopción, el sistema satisface las condiciones de *manos libres*, *oídos libres* y de dispositivo vestible. Sin embargo, requiere movimientos corporales activos para explorar el entorno, lo que compromete el criterio de *simplicidad* durante la marcha. La resolución espacial es limitada y no se contempla memoria temporal de las observaciones.

Bastones inteligentes

Santos et al. [2] proponen un bastón instrumentado con sensores ultrasónicos que retransmite la información de proximidad mediante vibración en el mango. Al incorporar los sensores directamente en el bastón blanco, el criterio de *manos libres* se cumple, dado que no se introduce un dispositivo adicional que ocupe las manos. El sistema es simple en su operación y no interfiere con la audición. La limitación estructural más relevante es que la detección queda restringida a la zona barrida por el bastón, sin cobertura vertical, lo que resulta en baja resolución espacial efectiva. Tampoco incorpora memoria temporal de obstáculos.

Dispositivos ultrasónicos portátiles

Petsiuk y Pearce [3] describen una pulsera ultrasónica de bajo costo que emite vibraciones al detectar objetos en un cono estrecho frente al brazo. El dispositivo es vestible y no ocupa las manos, aunque la posición en la muñeca requiere orientación activa del brazo. No interfiere con la audición y su operación es simple. No obstante, la resolución espacial es muy limitada: solo se detecta la distancia al obstáculo más próximo en la dirección del brazo, sin capacidad de representar la distribución espacial del entorno, y sin ninguna forma de acumulación temporal de observaciones.

2.1.3. Sistemas con RGB-D (percepción densa)

La incorporación de sensores de profundidad por estereoscopia activa en dispositivos de asistencia representa una continuidad natural de los ETA, orientada a superar la baja resolución de los sensores puntuales sin abandonar los criterios de adopción discutidos en la Section 2.1.1. Los sistemas revisados a continuación muestran distintos niveles de madurez dentro de esa misma línea evolutiva, desde retroalimentación directa sobre el fotograma instantáneo, pasando por integración de mapeo local con fusión inercial, hasta productos comerciales que consolidan el enfoque, como se discute en la Section 2.1.4.

Percepción instantánea sin memoria temporal

Yang et al. [14] proponen un sistema portátil basado en el sensor Intel RealSense R200 que detecta el área transitable y comunica la información al usuario mediante retroalimentación

sonora estereofónica. Su trabajo demuestra que la densidad espacial del sensor RGB-D se puede explotar en tiempo real sobre hardware embebido para mejorar significativamente la cobertura de detección respecto de sistemas ultrasónicos, siendo validado con participantes invidentes en entornos interiores y exteriores.

No obstante, el trabajo de Yang et al. no incorpora información inercial para compensar la inclinación del sensor respecto al plano horizontal, ni implementa una etapa de mapeo local. La representación final se obtiene directamente del fotograma instantáneo, por lo que toda información sobre obstáculos se pierde en cuanto estos abandonan el campo de visión de la cámara. Esto impide acumular un historial de observaciones que podría ser utilizado por una lógica de evitación de colisiones para anticipar obstáculos que, aunque ya no visibles en ese instante, siguen siendo relevantes para la trayectoria del usuario.

Sistemas con mapeo local e integración inercial

Un antecedente más completo es el sistema propuesto por Lee y Medioni [6], que integra una cámara RGB-D montada en la cabeza con una IMU de nueve grados de libertad, una grilla de ocupación 2D probabilística y un chaleco de retroalimentación vibrotáctil. El sistema estima la pose 6-DOF en tiempo real mediante un algoritmo híbrido que combina características visuales dispersas, nubes de puntos densas y la extracción del plano de suelo para reducir la deriva. Con esa estimación de pose construye un mapa incremental del entorno y planifica trayectorias seguras hacia un destino, transmitiendo la dirección al usuario a través del chaleco. El sistema fue validado con sujetos ciegos reales en entornos interiores no conocidos previamente y corre a aproximadamente 30 Hz sobre una laptop. La contribución central es demostrar que la combinación de RGB-D, IMU y grilla de ocupación resulta viable para asistencia portátil a ciegos en tiempo real; su limitación más evidente es la dependencia de hardware de cómputo no embebido (laptop).

Zhang et al. [7] avanzan sobre esta línea incorporando una IMU externa de mayor precisión (VN-100) y un algoritmo de odometría visual-inercial potenciado por profundidad denominado DVIO (*Depth-enhanced Visual-Inertial Odometry*). DVIO integra la extracción del plano de suelo con características visuales y mediciones inerciales en un marco de optimización de grafos para estimar la pose 6-DOF a 18 Hz sobre hardware embebido compacto. El sistema opera sobre una plataforma robótica de acompañamiento (*Robotic Navigation Aid*) y tiene como objetivo localizar la plataforma en un plano arquitectónico del edificio para guiar al usuario hacia un destino. Este trabajo muestra que la fusión profunda de datos de profundidad e inercia en un marco de optimización de grafos permite reducir la deriva de odometría en condiciones de iluminación variable y texturas pobres, comparado con los enfoques que tratan la IMU como entrada secundaria.

2.1.4. Producto comercial consolidado: *SuperBrain 1*

Los trabajos de investigación descritos en las secciones anteriores trazan una progresión técnica que encuentra su expresión comercial más completa en el dispositivo *SuperBrain 1*, desarrollado por la empresa estonia Seventh Sense OÜ [8]. Su existencia como producto comercial certificado (marcado CE) y validado con personas con discapacidad visual en múltiples países de Europa constituye evidencia directa de que la combinación de percepción de profundidad, procesamiento embebido y retroalimentación háptica vestibular puede dar lugar a un producto de uso real.

El dispositivo es un casco portado en la cabeza que integra cámaras estéreo de campo amplio (185° horizontal, 120° vertical), sensores de tiempo de vuelo (*Time-of-Flight*, ToF) para extender

el campo de detección lateral, y una cámara térmica que permite la percepción de personas y fuentes de calor. La interfaz de salida consiste en una matriz de puntos de presión sobre la frente del usuario (*haptic mat*), que transmite posición, movimiento y distancia de objetos de forma continua. A diferencia de los arreglos vibrotáctiles convencionales revisados en la Section 2.1.2, este mecanismo de presión genera una experiencia táctil más rica y precisa. Una función adicional, asistida por inteligencia artificial, implementa un *zoom* inteligente que permite enfocar selectivamente objetos de interés en la escena y representarlos con mayor resolución háptica, capacidad que supera lo previsto en el prototipo desarrollado en esta tesis.

Según la información pública del fabricante, el dispositivo no requiere el uso de las manos, no interfiere con la audición y se porta sobre el cuerpo. Estos atributos permiten analizarlo respecto de los criterios de adopción, aunque la simplicidad de uso requiere evidencia con usuarios y no se infiere únicamente de su descripción comercial. Su costo se encuentra en el orden de miles de euros y su desarrollo demandó más de diez años, magnitudes que muestran la diferencia de alcance entre un producto comercial certificado y un prototipo académico. En consecuencia, no se plantea una equivalencia funcional con esta tesis. El *SuperBrain 1* se utiliza como referencia de las capacidades que puede alcanzar una solución madura, mientras que este trabajo estudia un subconjunto acotado: percepción local, salida lógica compacta y ejecución en hardware embebido. Las funciones avanzadas del producto, como el *zoom* con inteligencia artificial y la cámara térmica, se proponen como líneas de trabajo futuro en el Capítulo 11.

2.1.5. Síntesis comparativa de los sistemas revisados

Los sistemas sin RGB-D revisados en la Section 2.1.2 comparten dos limitaciones fundamentales: en primer lugar, utilizan sensores puntuales o de muy bajo número de haces, lo que limita la representación de la distribución tridimensional de obstáculos en el campo visual del usuario. En segundo lugar, esas mediciones aisladas aportan poca información geométrica para estimar el movimiento del usuario por sí solas. Esto no impide obtener odometría mediante una IMU, una cámara u otro sensor adicional, pero los sistemas puntuales aquí revisados no incorporan esas fuentes ni una memoria espacial. Como resultado, el conocimiento sobre el entorno se descarta entre fotogramas y no puede utilizarse para inferir la presencia de obstáculos que hayan salido momentáneamente del campo sensado. Esta doble restricción dificulta la detección de obstáculos a altura de cabeza, la anticipación a geometrías complejas y la posibilidad de implementar lógicas de evitación de colisiones que operen sobre información acumulada.

En contraste, los sistemas con RGB-D incorporan cámaras de profundidad para aumentar la resolución espacial y, en algunos casos, añadir memoria temporal mediante mapeo local. Esta evolución tecnológica se expresa en la Tabla 2.1, que consolida la evaluación de todos los sistemas revisados en este capítulo, desde los basados en sensores puntuales hasta los de cámara de profundidad y el producto comercial *SuperBrain 1*, respecto de los criterios de Dakopoulos y Bourbakis [13] y las dos dimensiones técnicas adicionales. La tabla evidencia la progresión desde soluciones de baja resolución y sin memoria temporal hacia sistemas con percepción densa y acumulación de historial, con el nivel de inversión que cada etapa conlleva.

La clasificación es cualitativa y se limita a las capacidades descritas por cada fuente. Se considera resolución muy baja cuando el sistema entrega una única distancia o dirección, y alta cuando conserva una medición densa del campo observado. La memoria temporal se marca únicamente cuando el trabajo describe una representación persistente del entorno. La última fila corresponde al proyecto global, que integra el módulo de percepción y mapeo desarrollado en esta tesis con la interfaz háptica física implementada en la segunda etapa. Por eso los criterios de adopción se evalúan sobre el dispositivo completo, mientras que la resolución espacial y la memoria temporal provienen del aporte específico de esta tesis.

Tabla 2.1: Comparación de sistemas de asistencia sensorial para personas ciegas según los criterios de Dakopoulos y Bourbakis [13] y dos criterios técnicos adicionales. ✓: satisfecho; ~: parcialmente satisfecho; ×: no satisfecho. La fila «Proyecto global» considera la integración de las dos etapas del proyecto.

Sistema	Manos libres	Oídos libres	Vestible	Simple	Resolución espacial	Memoria temporal
Novich y Eagleman [1]	✓	✓	✓	~	baja	×
Santos et al. [2]	✓	✓	~	✓	muy baja	×
Petsiuk y Pearce [3]	~	✓	✓	✓	muy baja	×
Yang et al. [14]	✓	✓	✓	~	alta	×
Lee y Medioni [6]	✓	✓	✓	~	alta	✓
SuperBrain 1 [8]	✓	✓	✓	✓	alta	~
Proyecto global	✓	✓	✓	✓	media	✓

2.2. Mapeo local y evitación de obstáculos en robótica móvil

La revisión de los dispositivos de asistencia presentada en las secciones anteriores muestra que las limitaciones de resolución y de memoria temporal provienen, en parte, de no explotar las técnicas de mapeo local consolidadas en el campo de la robótica móvil. Esta sección revisa dichos enfoques y sus representaciones de ocupación probabilística, que constituyen el fundamento del módulo de mapeo desarrollado en esta tesis.

2.2.1. Mapas locales reactivos

En robótica móvil, los mapas locales se emplean en estrategias de navegación reactiva para acotar el área actualizada y evitar la dependencia inmediata de un mapa global. Brock y Khatib [15] argumentan que un mapa local centrado en el robot y actualizado a alta frecuencia permite reaccionar ante obstáculos dinámicos sin requerir la coherencia global de un mapa completo del entorno.

2.2.2. Representaciones de ocupación probabilística

La representación canónica para este tipo de mapas es la grilla de ocupación probabilística, introducida por Elfes [11] y formalizada en el marco bayesiano por Thrun, Burgard y Fox [12]. La formulación en escala de log-odds permite actualizar la probabilidad de ocupación de cada celda con costo computacional constante y acumular historial de observaciones, lo cual actúa como filtro de baja frecuencia sobre el ruido del sensor. El trazado de rayos mediante el algoritmo de Bresenham [16] complementa la actualización al marcar explícitamente como libres las celdas atravesadas por el haz antes del punto de impacto.

Existen representaciones volumétricas más expresivas, como OctoMap [17], que extienden la grilla de ocupación 2D al espacio tridimensional mediante octrees y permiten representar obstáculos a distintas alturas con resolución adaptativa. Sin embargo, su mayor fidelidad conlleva un costo computacional significativamente más alto que el de una grilla 2D plana. En plataformas de robótica móvil con cómputo embebido, el *costmap* local bidimensional de Nav2 [18] constituye una alternativa habitual: una grilla 2D de ocupación actualizada a partir de

sensores de profundidad, con capas configurables para inflación de obstáculos y filtrado temporal.

2.2.3. Representación 2D y compensación gravitacional

El mapa local desarrollado en esta tesis adopta los principios de la grilla de ocupación probabilística en escala de log-odds y utiliza una proyección 2D de la nube de puntos sobre el plano horizontal. La discriminación de obstáculos relevantes se resuelve antes de la proyección, mediante alineación gravitacional y filtrado por bandas de altura, conservando en la grilla únicamente la ocupación plana necesaria para la navegación local. De este modo se mantiene una representación compatible con las restricciones de cómputo embebido y con la interfaz háptica de baja resolución utilizada por el sistema.

La alineación gravitacional descrita se basa en la estimación instantánea del vector de gravedad a partir del acelerómetro [19], y es conceptualmente distinta de la odometría visual-inercial [20]-[22]: no integra velocidades ni construye una trayectoria, sino que estima roll y pitch instantáneos a partir de la componente estática de la aceleración.

2.2.4. Estimación del movimiento egocéntrico

El desplazamiento egocéntrico del mapa local ante traslaciones del usuario requiere una estimación del movimiento relativo de la cámara. El giroscopo permite estimar cambios de orientación, pero su integración aislada no observa la traslación y, por lo tanto, no constituye una odometría posicional completa. Para estimar trayectoria pueden emplearse el registro geométrico de nubes o la odometría RGB-D basada en características visuales y profundidad. Cada alternativa presenta compromisos diferentes entre costo computacional, sensibilidad a la geometría o textura de la escena y deriva [19], [23].

Los trabajos de asistencia revisados adoptan soluciones específicas: Lee y Medioni [6] emplean una estimación visual-inercial sobre una computadora portátil, mientras que Zhang et al. [7] usan DVIO con optimización de grafos sobre una plataforma robótica. En esta tesis se evaluó inicialmente el registro ICP de nubes y se adoptó finalmente `rgbd_odometry`, del paquete `rtabmap_odom` de RTAB-Map. El filtro Madgwick entrega una orientación obtenida de acelerómetro y giroscopo como entrada auxiliar; no se presenta esta configuración como una VIO estrechamente acoplada, porque RTAB-Map no optimiza conjuntamente los estados visuales e inerciales en el esquema utilizado. La Tabla 2.2 resume las alternativas relevantes y el papel de cada fuente.

2.3. Brecha identificada y posicionamiento de la tesis

Del análisis anterior se desprenden cuatro brechas concretas en el estado del arte:

1. **Resolución espacial insuficiente:** los ETA dominantes emplean sensores puntuales que no permiten representar la distribución tridimensional de obstáculos en el campo visual del usuario [2], [3], [13]. En particular, la ausencia de cobertura vertical impide detectar obstáculos a la altura de la cabeza, como ramas, cables o señales colgantes, que escapan al alcance del bastón blanco y requieren información espacial en el eje vertical.
2. **Ausencia de acumulación temporal de observaciones:** los sistemas basados en cámara de profundidad como el de Yang et al. [14] generan retroalimentación directamente desde el fotograma instantáneo, sin conservar historial de observaciones. La consecuencia más relevante no es la incapacidad de resolver oclusiones en profundidad, ya que una

Tabla 2.2: Alternativas consideradas para estimar el movimiento del mapa local y configuración adoptada con `rtabmap_odom`.

Modo	Fuente	Implementación	Compromisos
Orientación inercial	Acelerómetro y giróscopo	Madgwick [24]	Aporta orientación; no estima por sí sola la traslación
Registro geométrico	Nube de profundidad	<code>icp_odometry</code> de RTAB-Map [23]	Depende de geometría suficiente y de una inicialización válida; alternativa descartada
Odometría RGB-D adoptada	Imagen RGB, profundidad y orientación IMU	<code>rgbd_odometry</code> de <code>rtabmap_odom</code> [23]	Usa características y profundidad métrica; puede degradarse ante texturas pobres o movimientos rápidos

representación 2D con memoria temporal como la de esta tesis tampoco reconstruye zonas no visibles, sino la pérdida de información sobre obstáculos que han salido del campo de visión de la cámara. Un usuario que gira la cabeza o que avanza dejando un objeto fuera del encuadre pierde instantáneamente ese dato; sin acumulación temporal no es posible implementar lógicas de evitación que operen sobre el historial reciente del entorno, lo que limita estructuralmente la utilidad del sistema para anticipar colisiones con objetos que ya no están siendo observados en ese instante.

3. **Costo y accesibilidad del hardware:** el producto *SuperBrain 1* [8] muestra que la percepción densa y la realimentación háptica vestibular pueden integrarse en una solución comercial, pero su costo se encuentra en el orden de miles de euros y su maduración demandó más de diez años. Entre los trabajos académicos revisados, Lee y Medioni [6] emplean una computadora portátil y Zhang et al. [7] una plataforma robótica embebida. Estos antecedentes no permiten trasladar directamente sus resultados a un dispositivo corporal basado en Raspberry Pi 5; por ello, esta tesis debe caracterizar experimentalmente el costo computacional de su propia configuración.
4. **Necesidad de baja latencia:** las métricas temporales publicadas por los trabajos revisados no son directamente comparables, porque pueden referirse a frecuencia del sensor, tiempo de procesamiento o tasa de actualización, y no necesariamente al recorrido completo desde la adquisición hasta la advertencia. En un sistema de asistencia utilizado durante la marcha, la información debe actualizarse con suficiente rapidez para representar el obstáculo antes de que el usuario alcance su posición. Por ello, el retardo debe analizarse en relación con la velocidad de desplazamiento, la distancia de detección y el tiempo disponible para reaccionar, no solo con la frecuencia nominal de un nodo. Esta tesis incluye la latencia extremo a extremo entre las métricas del Capítulo 7; el umbral de aceptación se definirá antes de interpretar los resultados.

La presente tesis aborda estas cuatro brechas mediante la adaptación y validación de técnicas de percepción y mapeo local propias de la robótica móvil al dominio de la asistencia sensorial portátil. A diferencia de Lee y Medioni [6], cuyo sistema apunta a la navegación global con planificación de rutas, el enfoque adoptado en esta tesis se centra en la conciencia local de obstáculos y en la generación de una representación háptica compacta, priorizando la ejecución sobre hardware embebido de bajo consumo. Se adopta una representación de baja resolución de 5 filas $\times$ 10 columnas, compatible con arreglos de actuadores hápticos de baja complejidad y con las restricciones de ancho de banda y cómputo de plataformas embebidas, con un compromiso explícito entre resolución espacial y frecuencia de actualización en línea con la discusión de la Section 2.2.1. La cobertura vertical se obtiene durante el procesamiento 3D, antes de proyectar los obstáculos; la grilla 2D conserva ocupación sobre el plano horizontal y aporta memoria temporal, pero no mantiene información de altura. La plataforma Raspberry Pi 5 permite evaluar el costo computacional sobre hardware embebido, y la configuración de movimiento adoptada se describe en la Section 2.2.4.

Capítulo 3

Marco teórico

Los fundamentos se agrupan según la versión en la que cada herramienta se incorpora al sistema; la motivación y el alcance de v0, v1 y v2 se establecen en la Sección 1.3.3.

Este capítulo explica el significado de las representaciones y transformaciones utilizadas. La correspondencia concreta entre nodos, mensajes de entrada y mensajes de salida se muestra, para v0, v1 y v2, en las Figuras 4.4, 4.5 y 4.6 del capítulo de arquitectura. Esta separación permite desarrollar aquí los fundamentos sin repetir la topología de software y, a la vez, ofrece una guía para seguir cada concepto hasta el dato que efectivamente circula por ROS 2.

3.1. Fundamentos de la versión v0: percepción directa, ROS 2 y salida háptica

La versión v0 establece la cadena mínima del sistema: adquirir profundidad con la Intel RealSense D435i, reducir esa información a una representación compacta, convertirla a comandos de intensidad háptica y preparar su transmisión hacia el futuro hardware de actuación. Esta sección presenta los fundamentos teóricos que sostienen esa primera etapa, desde la sustitución sensorial hasta la comunicación serie embebida. El objetivo es mostrar cómo una medición densa del entorno puede convertirse en una señal discreta y de baja latencia sin perder la estructura espacial necesaria para la asistencia a la movilidad.

3.1.1. Sustitución sensorial y neuroplasticidad

El principio de sustitución sensorial fue establecido experimentalmente por Bach-y-Rita et al. [5]: una cámara proyectaba patrones de estimulación sobre la espalda del usuario mediante una matriz de 20×20 actuadores mecánicos, permitiendo a personas ciegas reconocer objetos y figuras a través del tacto. Este resultado demostró que el cerebro puede reorganizarse para interpretar señales de modalidades no originales cuando recibe información espacial consistente, fenómeno conocido como neuroplasticidad sensorial.

Para este trabajo, la relevancia de dicho antecedente es doble. En primer lugar, justifica que una escena visual pueda codificarse como estímulo háptico si se preserva una correspondencia espacial estable. En segundo lugar, orienta el diseño hacia representaciones compactas y repetibles: la interfaz no necesita reconstruir toda la imagen, sino entregar patrones táctiles consistentes que el usuario pueda aprender a interpretar.

3.1.2. Principio físico de la cámara Intel RealSense D435i

Los sensores RGB-D combinan información visual convencional con una medición de profundidad. En la familia Intel RealSense D400, y en particular en la D435i, la profundidad se obtiene mediante estereoscopía infrarroja activa: dos cámaras IR observan la escena y un proyector de textura facilita la correspondencia entre vistas en regiones con bajo contraste [4], [25]. El resultado es una imagen de profundidad $D(u, v)$ donde cada píxel codifica la distancia al plano de la cámara; los valores centinela $D = 0$ y $D = 65535$ indican ausencia de medición válida.

Bajo imágenes rectificadas, la correspondencia entre un píxel de la vista izquierda u_L y el píxel correspondiente de la vista derecha u_R define la disparidad $d = u_L - u_R$. Aplicando semejanza de triángulos en el sistema estereoscópico se obtiene:

$$Z = \frac{f B}{d}, \quad (3.1)$$

Aquí, Z es la profundidad del punto respecto del sistema estéreo, B es la separación entre los centros ópticos de las cámaras y f es la distancia focal expresada en píxeles [26], [27]. La ecuación 3.1 muestra que el error de profundidad crece al disminuir la disparidad, por lo que las mediciones lejanas son más sensibles al ruido y a errores de correspondencia.

La Figura 3.1 representa cada imagen rectificada únicamente sobre su eje horizontal. Las variables u_L y u_R son las coordenadas, en píxeles, donde se observa el mismo punto en las imágenes izquierda y derecha; su diferencia es la disparidad d . Los planos de imagen se dibujan delante de las cámaras porque se utiliza el modelo *pinhole* con un plano virtual. En la cámara física, el sensor se encuentra detrás del centro óptico y forma una imagen invertida; trasladar conceptualmente ese plano hacia delante permite representar una imagen derecha sin modificar la geometría de proyección. Por ello, el plano mostrado no representa un segundo sensor ni su posición física. La longitud rotulada f representa la distancia focal del modelo y, para combinarla con una disparidad medida en píxeles en la Ecuación 3.1, se utiliza su valor calibrado también en píxeles.

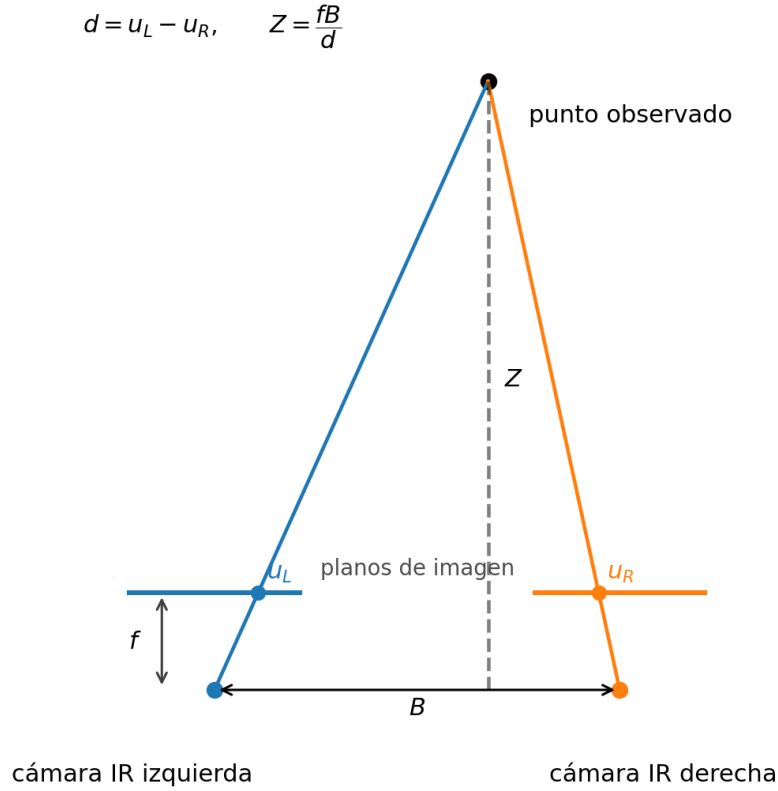


Figura 3.1: Geometría estereó rectificada. Un mismo punto se proyecta en las coordenadas horizontales u_L y u_R de los planos virtuales de imagen; su diferencia define la disparidad d .

La D435i publica, además de la imagen de profundidad, flujos de color, profundidad alineada a color, parámetros intrínsecos de cámara y señales inerciales. La distancia focal ya interviene en el cálculo estereó que la cámara realiza internamente para producir $D(u, v)$. Sin embargo, obtener la profundidad Z de un píxel no equivale a conocer la posición tridimensional completa del punto: para recuperar también sus coordenadas X e Y es necesario relacionar la posición (u, v) dentro de la imagen con la geometría calibrada de la cámara. Esta relación, y la razón por la que v0 y v1 utilizan entradas diferentes, se formalizan mediante el modelo *pinhole* de la subsección siguiente.

3.1.3. Modelo de cámara *pinhole* y proyección inversa

El modelo *pinhole* relaciona las coordenadas de imagen con las coordenadas espaciales. Un punto $\mathbf{P} = (X, Y, Z)^T$ expresado en el marco de la cámara se proyecta al píxel (u, v) mediante:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{Z} K \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}, \quad K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}. \quad (3.2)$$

En la ecuación 3.2, f_x y f_y son las distancias focales en píxeles, mientras que (c_x, c_y) es el centro de proyección. La operación inversa, necesaria para reconstruir una nube 3D a partir de una imagen de profundidad, queda:

$$X = \frac{(u - c_x) Z}{f_x}, \quad Y = \frac{(v - c_y) Z}{f_y}, \quad Z = D(u, v) \times 10^{-3} \text{ m}. \quad (3.3)$$

Los parámetros de K se obtienen del mensaje `sensor_msgs/CameraInfo`, lo que evita fijar intrínsecos a mano y mantiene coherencia con el perfil activo de resolución y calibración [26].

La diferencia entre ambas versiones no consiste, entonces, en que v0 prescinda de la calibración de la cámara, sino en dónde se utiliza. La RealSense la aplica internamente para entregar $D(u, v)$ a ambas versiones; solo v1 vuelve a consultar los intrínsecos porque necesita obtener X e Y . En v0, cada píxel permanece en coordenadas de imagen y se agrega directamente dentro de su ROI. En v1 puede procesarse un píxel cada s posiciones en ambas direcciones. Este *pixel stride* reduce aproximadamente la cantidad de proyecciones por un factor s^2 ; no debe confundirse con el *voxel grid*, que agrupa puntos después de proyectarlos al espacio tridimensional.

3.1.4. Limitaciones por superficies especulares e iluminación intensa

La estimación estéreo supone que los píxeles comparados en ambas cámaras observan el mismo punto físico mediante trayectorias ópticas directas y que su apariencia local es suficientemente similar. Esta hipótesis se cumple razonablemente bien en superficies difusas, pero se debilita en superficies especulares o semiespeculares, como pisos pulidos, baldosas brillantes, metal, vidrio o zonas húmedas. En esos casos la energía infrarroja emitida por el proyector de textura, o la luz proveniente de una fuente externa, se refleja de manera direccional y puede llegar a cada cámara desde caminos ópticos distintos.

El problema no es solamente la ausencia de datos. Si el algoritmo de correspondencia no encuentra textura confiable o detecta saturación, el resultado esperable es una medición inválida. Sin embargo, también puede ocurrir una correspondencia falsa pero internamente consistente. Un reflejo sobre el piso puede aparecer en ambas cámaras IR con posiciones compatibles con cierta disparidad d ; al aplicar la ecuación 3.1, el sistema interpreta esa disparidad como una profundidad real, aunque el patrón observado corresponda a una reflexión, a una imagen virtual o a luz parásita.

Al retroproyectar mediante la ecuación 3.3, esa medición se convierte en un punto 3D que no pertenece a un obstáculo físico. Para una interfaz háptica, este fenómeno tiende a manifestarse como falso positivo si la agregación por celda conserva una lectura cercana aislada. La mitigación requiere tratar estas lecturas como una limitación física del sensor y no como un error puramente algorítmico: descartar inválidos, exigir soporte espacial mínimo, incorporar persistencia temporal y verificar coherencia con el plano de suelo [4], [25].

3.1.5. ROS 2 como middleware de integración

ROS 2 organiza un sistema robótico como un grafo de nodos que intercambian mensajes tipados mediante tópicos, servicios y acciones [10]. En este trabajo se utiliza principalmente el patrón publicador-suscriptor: cada etapa produce una representación intermedia y la siguiente la consume sin depender de la implementación interna del productor. Este desacoplamiento permite separar adquisición, percepción, mapeo, visualización y salida al hardware.

La comunicación en ROS 2 se apoya en DDS. Esto habilita ejecutar el procesamiento principal en la Raspberry Pi y visualizar resultados en otra computadora dentro de la misma red, siempre que ambas participen del mismo dominio DDS. Además, los remapeos permiten adaptar nombres de tópicos en el lanzamiento sin introducir nodos adicionales. Esta propiedad es importante para la D435i, cuyo driver publica nombres nativos extensos, mientras que los nodos propios consumen interfaces normalizadas.

La teoría relevante para v0 no es solamente la existencia de tópicos, sino la elección de mensajes. Una imagen de profundidad se transporta como `sensor_msgs/Image`, los parámetros intrínsecos como `sensor_msgs/CameraInfo`, la grilla compacta como `DepthGrid` y la salida lógica

de intensidades como **HapticGrid**. Al definir mensajes explícitos se evita codificar semántica en arreglos sin contexto y se preserva trazabilidad entre nodos.

3.1.6. Reducción espacial de la imagen de profundidad

Una imagen de profundidad a resolución nativa contiene cientos de miles de píxeles por fotograma, pero esa densidad no puede trasladarse de forma directa a una interfaz háptica corporal. La limitación principal no es solamente computacional o de transmisión, sino perceptual: el usuario no puede distinguir de manera confiable estímulos hápticos arbitrariamente cercanos ni interpretar una matriz demasiado densa como si fuera una imagen visual. Por este motivo, la escena debe reducirse a una representación espacial de baja dimensión que conserve la ubicación aproximada y la urgencia de los obstáculos. En v0, esta reducción se realiza particionando la imagen en regiones de interés y calculando estadísticas por región.

La imagen de dimensiones $W \times H$ se divide en una grilla de $R \times C$ regiones de interés (ROI). Las fronteras de cada ROI se calculan mediante división entera uniforme:

$$x_c^{(k)} = \left\lfloor \frac{kW}{C} \right\rfloor, \quad k = 0, \dots, C; \quad y_r^{(j)} = \left\lfloor \frac{jH}{R} \right\rfloor, \quad j = 0, \dots, R. \quad (3.4)$$

La ecuación 3.4 distribuye el error de cuantización entre celdas, de modo que regiones adyacentes difieran a lo sumo en un píxel [28].

Para cada ROI(r, c) se recorren los píxeles válidos dentro del rango $[z_{\min}, z_{\max}]$ y se acumulan:

$$\bar{z}_{r,c} = \frac{1}{N_{r,c}} \sum_{(u,v) \in \text{ROI}_{r,c}} z_{u,v}, \quad z_{r,c}^{\min} = \min_{(u,v)} z_{u,v}, \quad N_{r,c} = \#\{\text{píxeles válidos}\}. \quad (3.5)$$

Junto con el máximo $z_{r,c}^{\max}$, estas estadísticas describen distintos aspectos de una región: el mínimo conserva la superficie medida más cercana, la media resume el conjunto de lecturas válidas y el máximo conserva la más lejana [17].

Por ejemplo, para las profundidades $[1,20; 0; 1,35; 1,10]$ m de una celda, el cero se descarta como medición inválida y se obtiene $N = 3$, $z^{\min} = 1,10$ m, $z^{\max} = 1,35$ m y $\bar{z} \simeq 1,22$ m. Este ejemplo muestra cómo varios píxeles de la imagen quedan resumidos en las estadísticas de una única celda de **DepthGrid**; la selección del estadístico empleado para generar la intensidad se presenta como decisión de diseño en la Sección ??.

La Figura 3.2 muestra esta reducción sobre un fotograma real. La misma ROI se destaca en la imagen original, en su ampliación y en la posición correspondiente de la grilla. Los píxeles negros representan mediciones inválidas o fuera del rango operativo. Para visualizar la grilla completa se representa el campo mínimo de cada celda, mientras que el recuadro de la ROI informa también cantidad, media y máximo. La coincidencia indicada en la figura verifica que esos valores fueron recalculados desde la imagen emparejada y coinciden con el mensaje **DepthGrid** publicado.

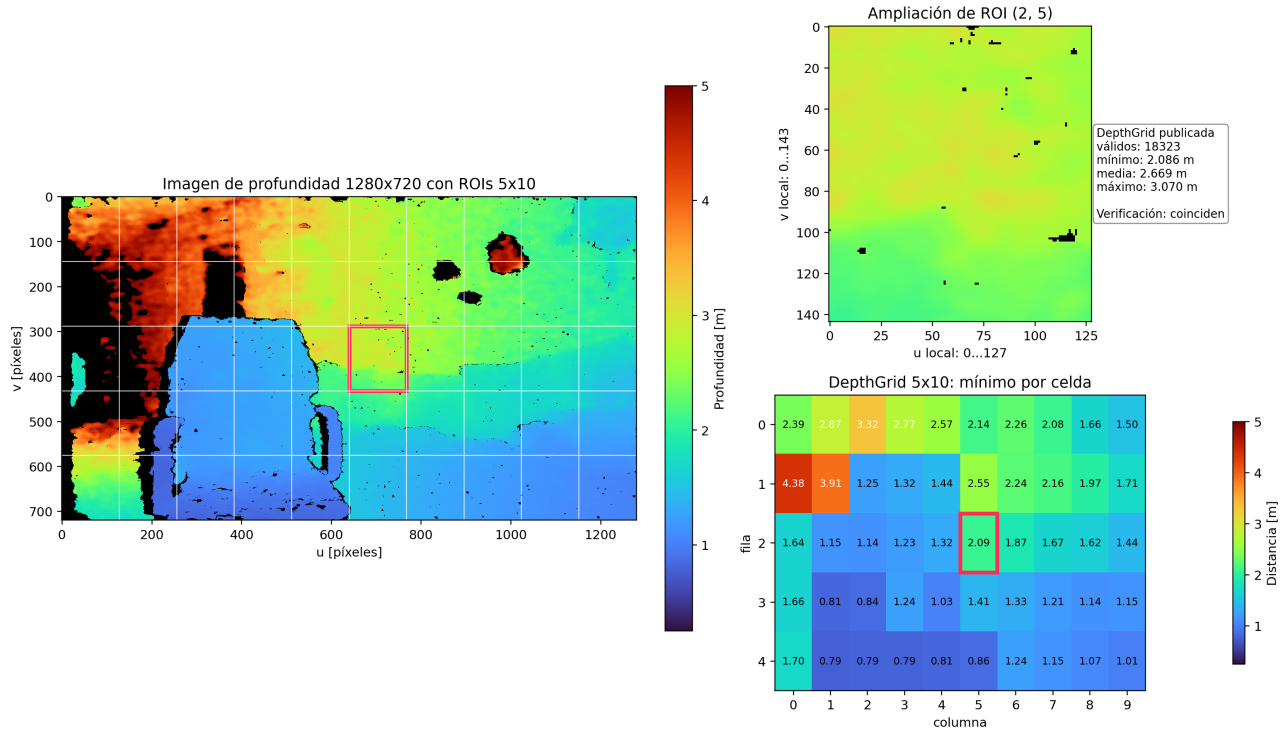


Figura 3.2: Reducción espacial de una imagen de profundidad de 1280×720 píxeles a una DepthGrid de 5 filas por 10 columnas. La captura proviene de la ejecución de v0.

3.1.7. Codificación angular de la nube filtrada

Aunque v0 reduce directamente la imagen de profundidad en ROIs de imagen, las versiones v1 y v2 construyen la grilla a partir de una nube de obstáculos ya filtrada. En ese caso, la partición horizontal puede definirse en ángulo, no solo en coordenada de imagen. Si un punto tiene coordenadas (x, z) en el plano horizontal de la cámara, su ángulo lateral es:

$$\beta = \text{atan2}(x, z). \quad (3.6)$$

La apertura horizontal activa se divide en columnas, y cada punto se asigna a la columna correspondiente según β . La distancia usada para codificar proximidad es radial:

$$d = \sqrt{x^2 + z^2}. \quad (3.7)$$

La ecuación 3.7 evita subestimar obstáculos laterales: usar solo z puede hacer que dos obstáculos con distinta separación lateral parezcan equivalentes si comparten la misma componente frontal.

La codificación implementada conserva cinco bandas métricas sobre el eje vertical. En el eje horizontal acumula primero sobre una grilla angular intermedia de mayor resolución que cubre la apertura configurada del sensor y luego reduce únicamente la apertura activa a las diez columnas publicadas. Así, modificar la apertura redistribuye los sectores visibles sin alterar el tamaño de la interfaz. En modo angular, la fila cero corresponde al límite vertical $y_{\text{mín}}$ y la última fila al límite $y_{\text{máx}}$. El modo métrico histórico invierte ese orden y no se utiliza como referencia para la configuración angular.

La Figura 3.3 vincula la imagen de profundidad de entrada con una nube real publicada por `depth_obstacle_filter`. La imagen permite reconocer la escena antes de la proyección y del filtrado geométrico; no se divide en ROIs porque, a diferencia de v0, en v1 la asignación se realiza sobre la nube 3D. En la vista superior, las líneas discontinuas delimitan la apertura

angular intermedia de $\pm 70^\circ$, discretizada internamente en sectores de 1° . Las líneas continuas delimitan las diez columnas de salida dentro de la apertura nominal de $\pm 45^\circ$; el color de cada punto indica la columna a la que fue asignado. La línea punteada roja marca el límite inferior efectivo de -54° . Esta asimetría surge porque la conversión a índice entero implementada trunca hacia cero los valores inmediatamente inferiores al límite nominal y los asigna a la columna 0. En la vista lateral, el intervalo $[y_{\min}, y_{\max}] = [-0,20, 1,50]$ m se divide en cinco bandas métricas: la fila 0 comienza en $y_{\min}$ y la fila 4 termina en $y_{\max}$. Finalmente, la matriz muestra la distancia radial mínima conservada en cada celda. Por ello puede aparecer una distancia ligeramente mayor que el límite aplicado a z : el filtrado limita la componente frontal, mientras que la matriz almacena $d = \sqrt{x^2 + z^2}$.

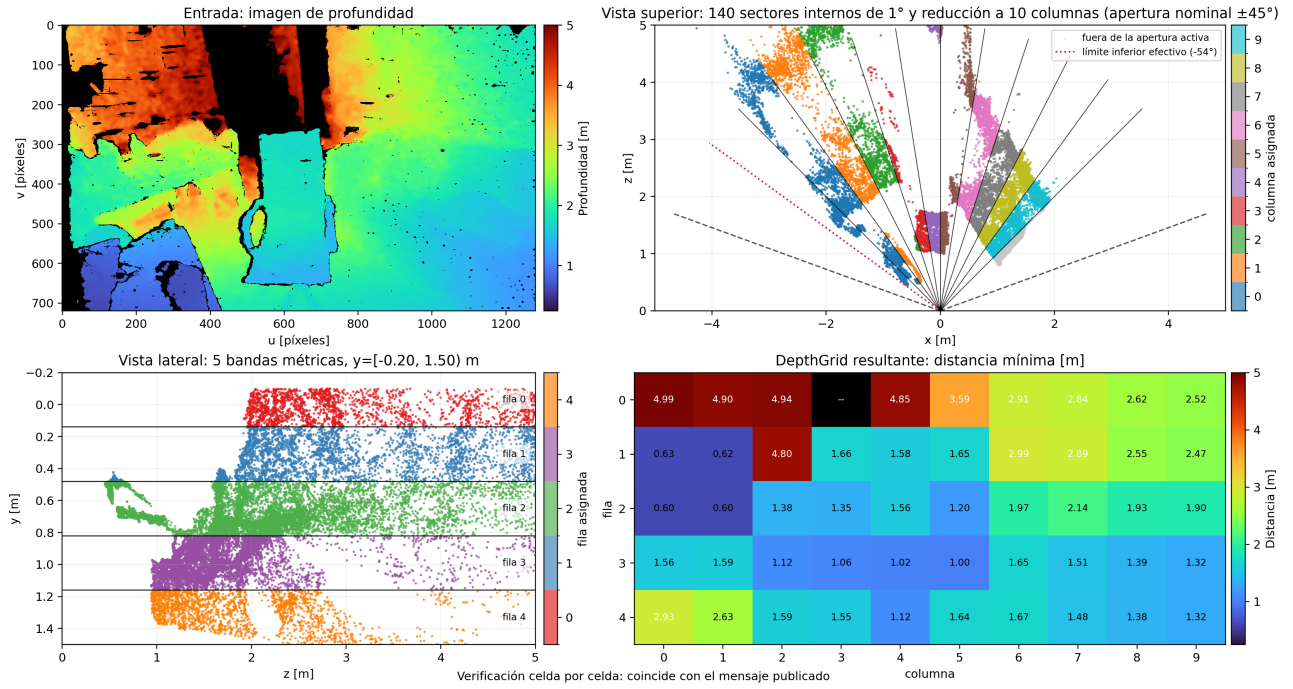


Figura 3.3: Imagen de profundidad de entrada y codificación angular y vertical de su nube de obstáculos filtrada en una *DepthGrid* de 5 filas por 10 columnas. La reconstrucción fuera de línea coincide celda por celda con las estadísticas publicadas por *obstacle_grid.encoder*.

3.1.8. Mapeo de distancia a comando de intensidad

Para traducir la distancia por celda a una señal háptica se emplea una función lineal invertida y saturada. La proximidad del obstáculo debe producir mayor intensidad, con saturación en los extremos del rango útil:

$$I_i = \begin{cases} I_{\max} & \text{si } z_i^{\min} \leq z_{\min}, \\ 0 & \text{si } z_i^{\min} \geq z_{\max}, \\ \frac{z_{\max} - z_i^{\min}}{z_{\max} - z_{\min}} \cdot I_{\max} & \text{en otro caso.} \end{cases} \quad (3.8)$$

En la ecuación 3.8, $z_{\min}$ y $z_{\max}$ delimitan exclusivamente el rango de conversión háptica, configurado entre 0,25 y 3,0 m; $I_{\max}$ es la intensidad máxima e I_i es la intensidad lógica asociada a la celda i . Estos límites no coinciden con el intervalo perceptivo completo. Los codificadores de v_0 y v_1 aceptan mediciones entre 0,25 y 5,0 m: las que quedan fuera de ese intervalo se

descartan y, si una celda no conserva ninguna muestra, publica contador nulo y estadísticas NaN. `haptic_grid_generator` trata esa celda como inactiva y le asigna intensidad cero. En cambio, una distancia válida comprendida entre 3,0 y 5,0 m se conserva en `DepthGrid`, pero la saturación de la Ecuación 3.8 produce intensidad cero. En la implementación actual esa celda conserva el indicador `active` porque contiene una medición válida, aunque el comando de salida sea nulo.

La intensidad lógica no es todavía el ciclo de trabajo aplicado a un motor. La interfaz UART posterior limita I_i al intervalo $[0, 100]$ y la escala mediante el máximo configurado $D_{\text{máx}}$:

$$D_i = \begin{cases} \left\lfloor \frac{\text{round}(\text{sat}_{[0,100]}(I_i)) D_{\text{máx}}}{100} \right\rfloor, & \text{si la celda está activa,} \\ 0, & \text{si la celda está inactiva.} \end{cases} \quad (3.9)$$

Esta segunda operación no vuelve a interpretar la distancia: solo adapta el comando ya calculado al límite de salida y lo prepara para su serialización.

La Figura 3.4 representa esta diferencia entre el rango perceptivo y el rango de conversión háptica. El tramo constante a intensidad máxima por debajo de 0,25 m describe la saturación matemática de la función, aunque esas mediciones son descartadas normalmente por los nodos perceptivos antes de alcanzar esta etapa.

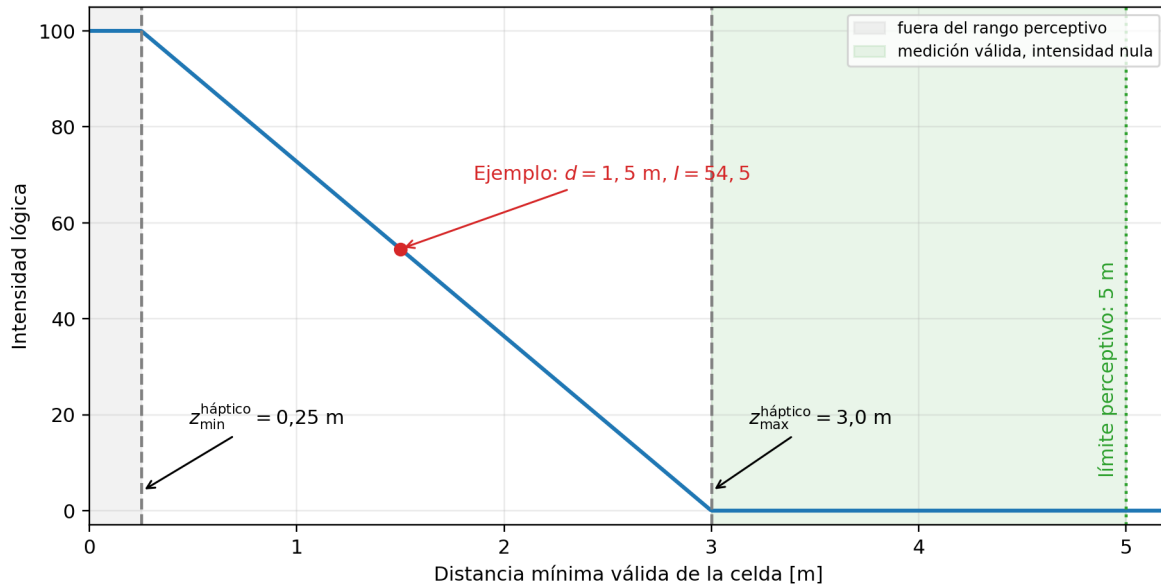


Figura 3.4: Conversión de la distancia mínima válida en intensidad lógica. Las mediciones válidas entre 3 y 5 m permanecen en `DepthGrid`, pero producen intensidad nula; una celda sin mediciones válidas contiene estadísticas NaN y queda inactiva.

3.1.9. Comunicación serie embebida

La transmisión prevista entre la computadora de placa simple y el futuro controlador de actuación requiere definir un protocolo de enlace. UART (*Universal Asynchronous Receiver-Transmitter*) es una opción habitual en sistemas embebidos por su disponibilidad y simplicidad. En modo 8N1, cada byte requiere 10 bits físicos: un bit de inicio, ocho bits de datos y un bit de parada. Por lo tanto, para una trama de longitud L_{trama} y baudrate B :

$$t_{\text{tx}} = \frac{10 \cdot L_{\text{trama}}}{B}. \quad (3.10)$$

La ecuación 3.10 permite verificar que la transmisión no comprometa el período del lazo háptico.

Existen dos enfoques de codificación. En ASCII, los valores se representan como caracteres decimales separados por espacios y terminados con `\n`; es legible y fácil de depurar, pero menos eficiente. En binario compacto, cada intensidad puede codificarse en un `uint8`; al agregar longitud y CRC-8 se obtiene una trama más robusta y eficiente [29]. En la configuración actual, el formato ASCII de tipo `M D0 D1 ... DN-1\n` resulta suficiente para una matriz de 5 filas $\times$ 10 columnas a 115200 bps, mientras que un formato binario queda como alternativa natural para matrices de mayor densidad.

En esta tesis se implementan la construcción y la transmisión de la trama, pero no su recepción ni la conversión de los valores transmitidos en estímulos físicos. El microcontrolador receptor, la electrónica de potencia y los actuadores pertenecen a la segunda etapa del proyecto.

La separación funcional es, por lo tanto, la siguiente: `haptic_grid_generator` calcula la intensidad lógica, `hardware_manager` la limita, escala y serializa, y `output_viewer` permite verificar la matriz sin depender del hardware físico.

El flujo entre estos componentes se muestra en la Figura 4.4: la `DepthGrid` es transformada por `haptic_grid_generator` en una `HapticGrid`, que constituye el contrato común de salida. A partir de ese mensaje, `hardware_manager` construye la trama UART descrita en esta sección, mientras que `output_viewer` permite inspeccionar directamente la misma matriz lógica sin intervenir en su cálculo ni en su transmisión. Las responsabilidades y las interfaces de ambas ramas se detallan en las Secciones 4.10 y 4.11.

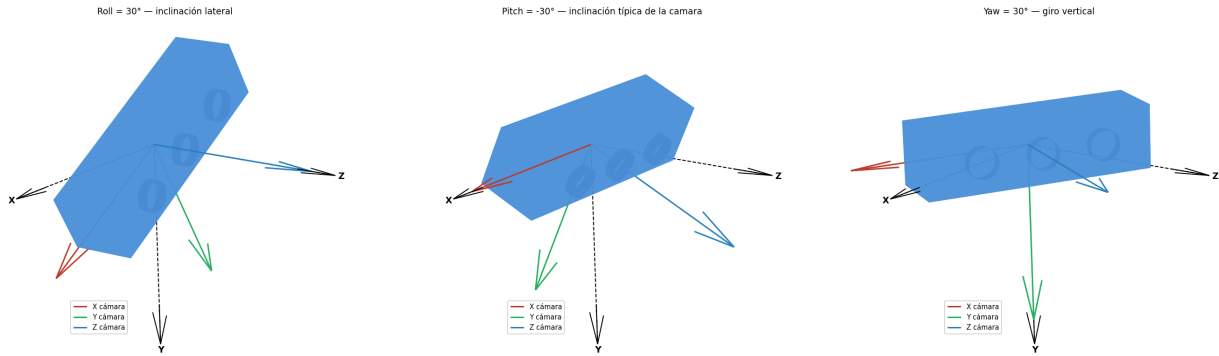
3.2. Fundamentos de la versión v1: orientación, gravedad y suelo

La versión v1 parte de la solución funcional establecida en v0 y conserva su salida compacta, pero mejora la interpretación previa de las mediciones al modificar la fuente de la grilla: en lugar de codificar profundidad cruda, primero proyecta la imagen a una nube 3D, la alinea con la gravedad y separa suelo, obstáculos y una región superior excluida. La teoría de esta etapa combina representaciones de orientación, filtrado inercial y estimación robusta de planos. En esta versión no se utiliza Madgwick de forma deliberada: la decisión de diseño es estimar solamente la vertical necesaria para remover el suelo, evitando incorporar una estimación completa de orientación cuando no aporta a la salida háptica y puede aumentar latencia, dependencias y costo computacional.

3.2.1. Ángulos de cámara y convención óptica

La orientación de la cámara puede describirse mediante ángulos de Euler: *roll* (ϕ), *pitch* (θ) y *yaw* (ψ). En la convención óptica de la D435i, *x* apunta hacia la derecha de la imagen, *y* hacia abajo y *z* hacia adelante. En esa convención, roll corresponde a la rotación alrededor de *z*, pitch alrededor de *x* e yaw alrededor de *y*, como se muestra en la Figura 3.5.

La Figura 3.5 también permite distinguir dos funciones diferentes. Roll y pitch modifican la relación entre la nube y el plano de suelo, por lo que deben compensarse antes de segmentar. En cambio, yaw rota la cámara alrededor de la vertical y no cambia qué puntos pertenecen al suelo; su importancia aparece en v2, cuando se acumulan observaciones en un marco común.



(a) Roll = 30°: rotación alrededor de z . (b) Pitch = -30°: rotación alrededor de x . (c) Yaw = 30°: rotación alrededor de y .

Figura 3.5: Ángulos de Euler en la convención óptica de la cámara. Cada rotación se representa alrededor de un eje fijo del sensor.

3.2.2. Singularidad de Euler y cuaterniones

Los ángulos de Euler presentan una singularidad conocida como *gimbal lock*: cuando $\theta = \pm 90^\circ$, dos ejes de rotación quedan alineados y se pierde un grado de libertad. La Figura 3.6 muestra una configuración alcanzable cuando la cámara apunta casi verticalmente hacia abajo.

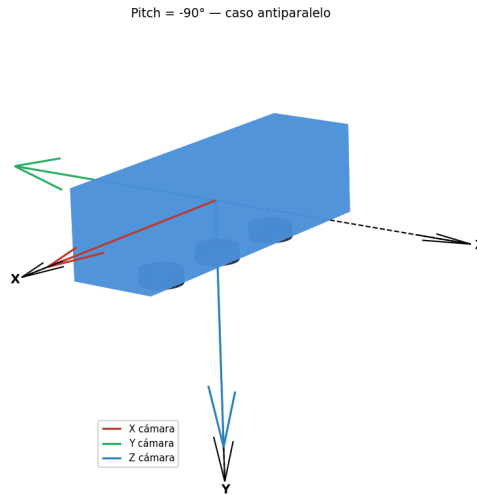


Figura 3.6: Singularidad *gimbal lock*: con pitch = -90° la cámara apunta verticalmente hacia abajo y los ejes de roll y yaw quedan alineados.

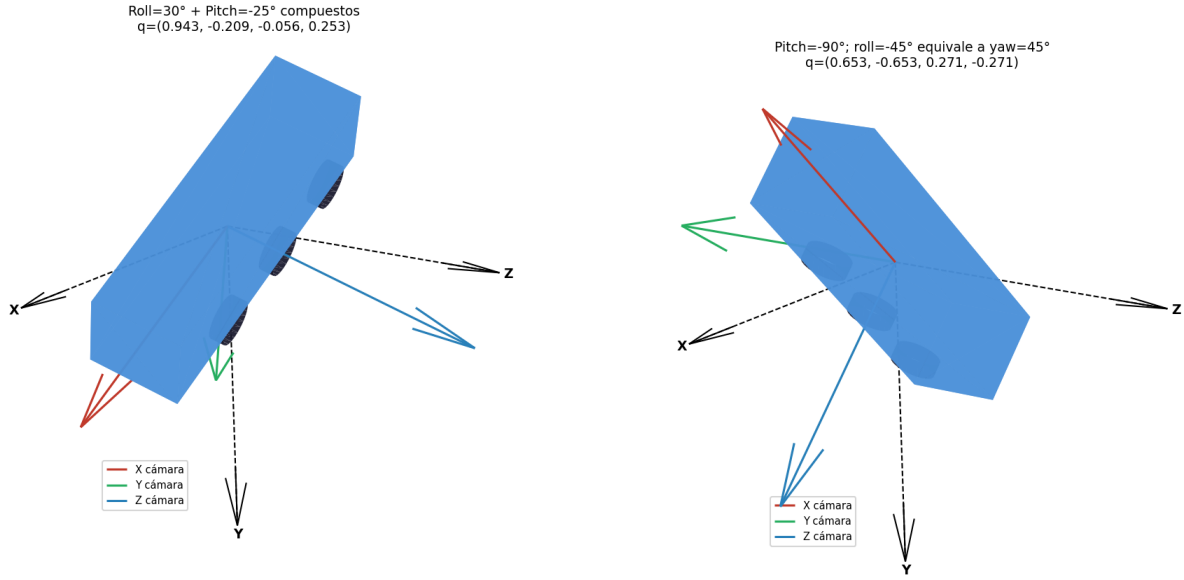
Para evitar esa singularidad se usa una representación mediante cuaterniones unitarios. Un cuaternión se define como:

$$q = q_w + q_x \mathbf{i} + q_y \mathbf{j} + q_z \mathbf{k}, \quad (3.11)$$

donde q_w es la parte escalar y $(q_x, q_y, q_z)^T$ la parte vectorial. Si $\|q\| = 1$, representa una rotación de ángulo θ alrededor de un eje unitario $\hat{\mathbf{n}}$:

$$q = \left(\cos \frac{\theta}{2}, \hat{\mathbf{n}} \sin \frac{\theta}{2} \right). \quad (3.12)$$

La rotación de un punto $\mathbf{p}$ se expresa como $\mathbf{p}' = q \otimes (0, \mathbf{p}) \otimes q^*$, donde q^* es el conjugado. La Figura 3.7 muestra las poses resultantes junto con el cuaternión unitario que las representa: a la izquierda, una composición de roll y pitch; a la derecha, una configuración singular para Euler donde distintas ternas angulares pueden describir la misma orientación física.



(a) Roll y pitch compuestos en un único cuaternión.

(b) Orientación en la singularidad de Euler.

Figura 3.7: Ventajas de la representación mediante cuaterniones para orientaciones compuestas y configuraciones singulares.

3.2.3. Alineación gravitacional con acelerómetro

La IMU de la D435i incluye acelerómetro y giróscopo de tres ejes [4]. En v1, el acelerómetro se usa para estimar la dirección de la gravedad. Esta elección evita integrar el giróscopo o ejecutar un filtro de orientación completo: para detectar suelo alcanza con conocer roll y pitch respecto de la vertical, mientras que yaw no modifica la altura de los puntos ni la coplanaridad del piso. En reposo, en el frame óptico de la cámara, una cámara horizontal mide aproximadamente $\mathbf{a} = (0, -g, 0)$, como se muestra en la Figura 3.8.

A partir de $\mathbf{a} = (a_x, a_y, a_z)^T$ pueden estimarse roll y pitch sin integración temporal:

$$\phi = \text{atan2}(a_y, a_z), \quad \theta = \text{atan2}\left(-a_x, \sqrt{a_y^2 + a_z^2}\right). \quad (3.13)$$

La ecuación 3.13 no estima yaw, ya que la gravedad no contiene información sobre rotación alrededor de la vertical. Estas expresiones permiten interpretar la medición, pero la implementación no calcula ambos ángulos para encadenar sus rotaciones. Para construir el marco alineado a gravedad se calcula la rotación mínima que lleva el vector de aceleración normalizado $\hat{\mathbf{a}}$ al vector canónico $\hat{\mathbf{b}} = (0, -1, 0)$. En forma proporcional:

$$q_w = 1 + \hat{\mathbf{a}} \cdot \hat{\mathbf{b}}, \quad (3.14)$$

$$\mathbf{q}_v = \hat{\mathbf{a}} \times \hat{\mathbf{b}}. \quad (3.15)$$

Luego se normaliza $q \leftarrow q/\|q\|$. La Figura 3.9 muestra el caso identidad y el caso antiparalelo, donde el eje queda indeterminado y debe resolverse de forma explícita.

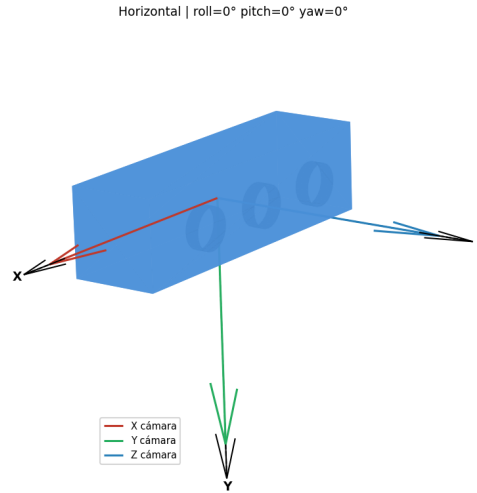


Figura 3.8: Cámara D435i en posición horizontal. El acelerómetro mide $(0, -g, 0)$ en la convención óptica usada por el pipeline.

3.2.4. Filtrado inercial local para la dirección de gravedad

El acelerómetro no mide únicamente gravedad. Durante la marcha registra aceleraciones lineales de la cabeza, vibraciones y golpes [19], [30]. Por eso, antes de usarlo como referencia vertical se aplica un filtro IIR pasa bajos de primer orden:

$$\mathbf{a}_f[k] = \alpha \mathbf{a}[k] + (1 - \alpha) \mathbf{a}_f[k - 1]. \quad (3.16)$$

La constante usada en el código, $\alpha \approx 0,136$, corresponde a una frecuencia de corte aproximada de 5 Hz con una tasa de muestreo cercana a 200 Hz [31]. Además, se descartan muestras cuya norma supera $2,5g$ y se calcula un indicador diagnóstico de dinámica cuando la norma filtrada se aleja más de $0,5 \text{ m/s}^2$ de g . El indicador informa que la medición puede contener una componente dinámica significativa, pero en la versión implementada no bloquea ni reemplaza por sí solo la rotación estimada. Este esquema permite estabilizar la vertical sin depender de la odometría ni de un filtro de orientación completo, manteniendo el procesamiento de v1 acotado para tiempo real.

3.2.5. Integración de giróscopo y deriva de yaw

El giróscopo mide velocidad angular. Si se integra la componente asociada al eje vertical, el yaw evoluciona como:

$$\psi(t) = \psi(t - \Delta t) + \omega_y(t) \Delta t. \quad (3.17)$$

La ecuación 3.17 permite entender la deriva: un sesgo constante del giróscopo produce error angular acumulativo [19]. En v1, este yaw no es necesario para segmentar suelo; en v2, la orientación horizontal se obtiene de la odometría RGB-D, que recibe la orientación filtrada de la IMU como entrada auxiliar.

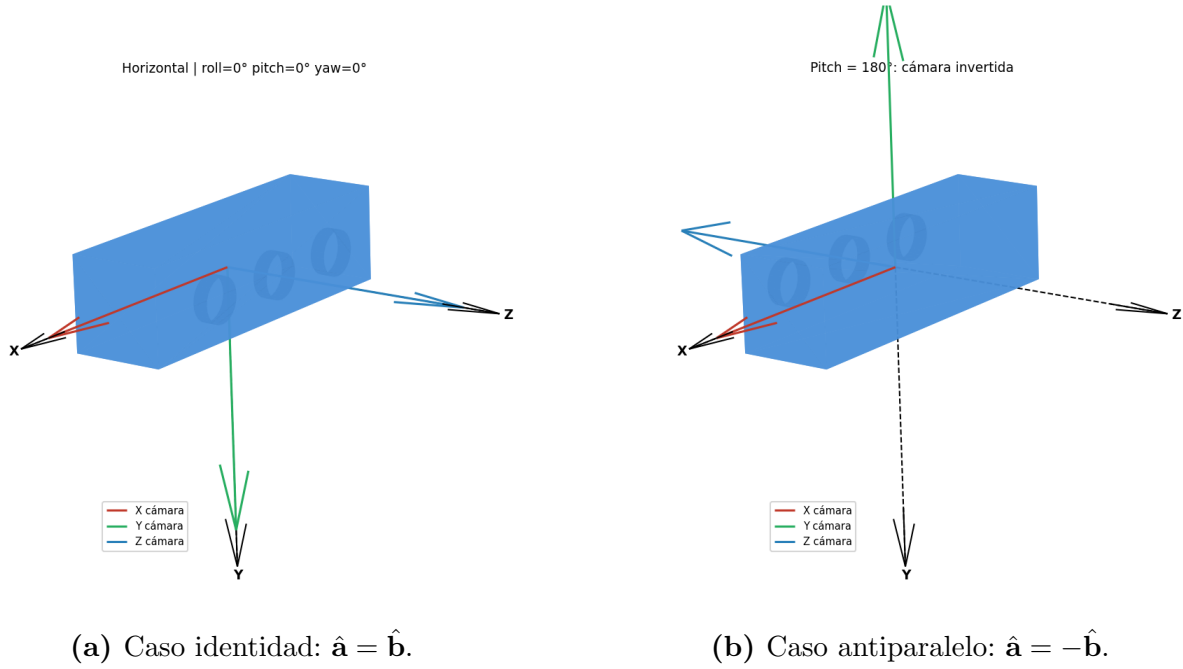


Figura 3.9: Casos límite en el cálculo de la rotación mínima entre el vector de aceleración y la vertical canónica.

3.2.6. Detección del plano del suelo

Una vez que la nube está alineada a gravedad, el suelo aparece como un conjunto de puntos coplanares a una altura aproximadamente constante. Un plano se representa mediante:

$$\mathbf{n} \cdot \mathbf{p} + d = 0, \quad (3.18)$$

donde $\mathbf{n}$ es la normal unitaria y d la distancia con signo al origen. La distancia con signo de un punto $\mathbf{q}$ al plano es:

$$\delta(\mathbf{q}) = \mathbf{n} \cdot \mathbf{q} + d. \quad (3.19)$$

La ecuación 3.19 permite clasificar puntos según su separación respecto del plano estimado.

3.2.7. Estimación adaptativa de ruido

El ruido de profundidad depende de la distancia y de la reflectividad. Para adaptar la tolerancia de inliers, se estima la distancia media al vecino más cercano sobre un subconjunto de la nube [28]:

$$\bar{d}_{NN} = \frac{1}{m} \sum_{i=1}^m \min_{j \neq i} \|\mathbf{p}_i - \mathbf{p}_j\|. \quad (3.20)$$

La desviación estándar de esa distribución, σ_{NN} , se usa como indicador empírico de la variabilidad espacial de la nube. No equivale estrictamente a la desviación de residuos gaussianos respecto de un plano: también depende de la densidad, las discontinuidades y el submuestreo. Por eso, el factor tres constituye una heurística de adaptación y no un intervalo de confianza probabilístico:

$$\varepsilon = 3 \sigma_{NN}. \quad (3.21)$$

En la configuración automática, el resultado se restringe al intervalo de 0,01 a 0,10 m; si no puede estimarse se emplea 0,03 m. Durante la evaluación de cada punto, esta tolerancia se combina además con un término proporcional al alcance, como se desarrolla en la Ecuación 3.31.

3.2.8. Submuestreo por voxel grid

Una nube a plena resolución puede contener cientos de miles de puntos. Como el conteo de inliers de RANSAC escala linealmente con la cantidad de puntos, una estrategia habitual de reducción es el submuestreo por *voxel grid*: el espacio se divide en celdas cúbicas de lado s y cada punto $\mathbf{p}_i = (x_i, y_i, z_i)$ se asigna a la clave discreta

$$\mathbf{k}_i = \left(\left\lfloor \frac{x_i}{s} \right\rfloor, \left\lfloor \frac{y_i}{s} \right\rfloor, \left\lfloor \frac{z_i}{s} \right\rfloor \right).$$

Los puntos que caen en una misma celda $V_{\mathbf{k}}$ se reemplazan por un único representante. En la implementación analizada, dicho representante es el promedio aritmético por coordenada,

$$\bar{\mathbf{p}}_{\mathbf{k}} = \frac{1}{N_{\mathbf{k}}} \sum_{\mathbf{p}_i \in V_{\mathbf{k}}} \mathbf{p}_i,$$

es decir, el centroide de las muestras del voxel [17]. Como se esquematiza en la Figure 3.10, solo los puntos contenidos en una celda ocupada aportan al representante de esa celda. No se conserva el mínimo, el máximo ni la profundidad más cercana: el filtro suaviza localmente la nube y reduce la cantidad de puntos. Esta propiedad es apropiada para estimar geometría dominante, como el plano del suelo, pero puede atenuar detalles pequeños si se usara como representación final de obstáculos.

El filtro está implementado como alternativa, pero se encuentra desactivado en la configuración operacional analizada. En esa configuración se conserva la nube proyectada y se limita la cantidad de candidatos de RANSAC dentro de la banda de alturas. Por lo tanto, el *voxel grid* no debe interpretarse como una etapa obligatoria del recorrido activo.

3.2.9. RANSAC y refinamiento por mínimos cuadrados

En una escena real, el plano del suelo coexiste con obstáculos, paredes y ruido. RANSAC (*Random Sample Consensus*) estima modelos paramétricos en presencia de valores atípicos [12]. Para planos, cada iteración muestrea tres puntos, ajusta un candidato y cuenta inliers con distancia menor que ε . Si la fracción de inliers es ρ , la probabilidad de fallar en K iteraciones es:

$$P_{\text{fail}} = (1 - \rho^3)^K. \quad (3.22)$$

Esta relación permite controlar el compromiso entre robustez y tiempo de cómputo.

El modelo no requiere que el suelo coincida con un valor constante exacto de altura. Si el piso presenta una pendiente moderada, el conjunto de puntos puede seguir explicándose mediante un plano inclinado; la condición relevante para RANSAC es la distancia de cada muestra al plano candidato, no la igualdad de una coordenada particular. La Figure 3.11 ilustra esta idea sobre un corte bidimensional, usando la convención geométrica adoptada en el sistema.

Luego de seleccionar los inliers, el plano se refina por mínimos cuadrados. Se minimiza:

$$J(\mathbf{n}, d) = \sum_{i=1}^n (\mathbf{n}^T \mathbf{p}_i + d)^2. \quad (3.23)$$

Derivando respecto de d se obtiene el centroide:

$$\bar{\mathbf{p}} = \frac{1}{n} \sum_{i=1}^n \mathbf{p}_i, \quad d^* = -\mathbf{n}^T \bar{\mathbf{p}}. \quad (3.24)$$

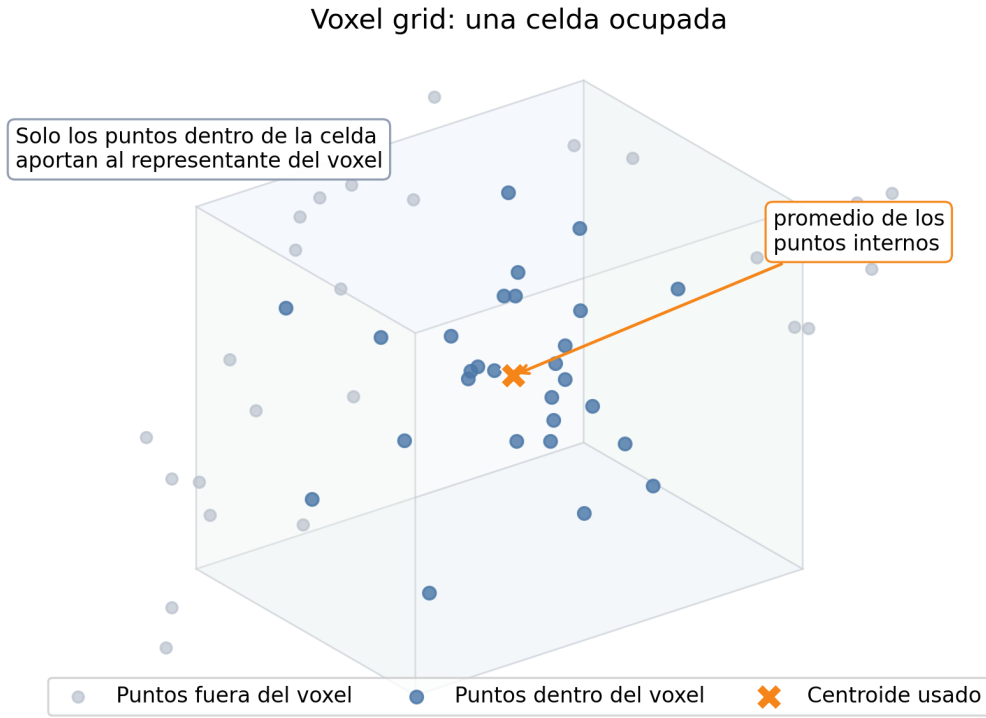


Figura 3.10: Esquema tridimensional de submuestreo por *voxel grid*. Los puntos originales dentro de cada celda ocupada se reemplazan por un único representante ubicado en el centroide de las muestras. Elaboración propia.

El problema se reduce a hallar la dirección de menor varianza de la matriz de covarianza:

$$C = \frac{1}{n} \sum_{i=1}^n (\mathbf{p}_i - \bar{\mathbf{p}})(\mathbf{p}_i - \bar{\mathbf{p}})^T. \quad (3.25)$$

La normal óptima es el autovector asociado al menor autovalor [26, App. A4]:

$$\mathbf{n}^* = \mathbf{q}_3, \quad d^* = -\mathbf{n}^{*T} \bar{\mathbf{p}}. \quad (3.26)$$

Para matrices 3×3 simétricas, el cálculo puede realizarse mediante iteraciones de Jacobi. Cada iteración aplica una rotación ortogonal $R_{ij}(\theta)$:

$$C_{k+1} = R_{ij}(\theta)^T C_k R_{ij}(\theta), \quad (3.27)$$

con:

$$\theta = \frac{1}{2} \text{atan2}(2(C_k)_{ij}, (C_k)_{ii} - (C_k)_{jj}). \quad (3.28)$$

3.2.10. Robustez ante suelo parcialmente visible

Cuando la cámara apunta al frente, el suelo puede ocupar una fracción pequeña de la nube. Una métrica de calidad respecto del total de puntos penalizaría escenas donde el piso es visible solo en una banda inferior. Para resolverlo se restringe el muestreo a una banda candidata:

$$\mathcal{B} = \{i : Y^* - \delta_l \leq Y_i \leq Y^* + \delta_h\}, \quad (3.29)$$

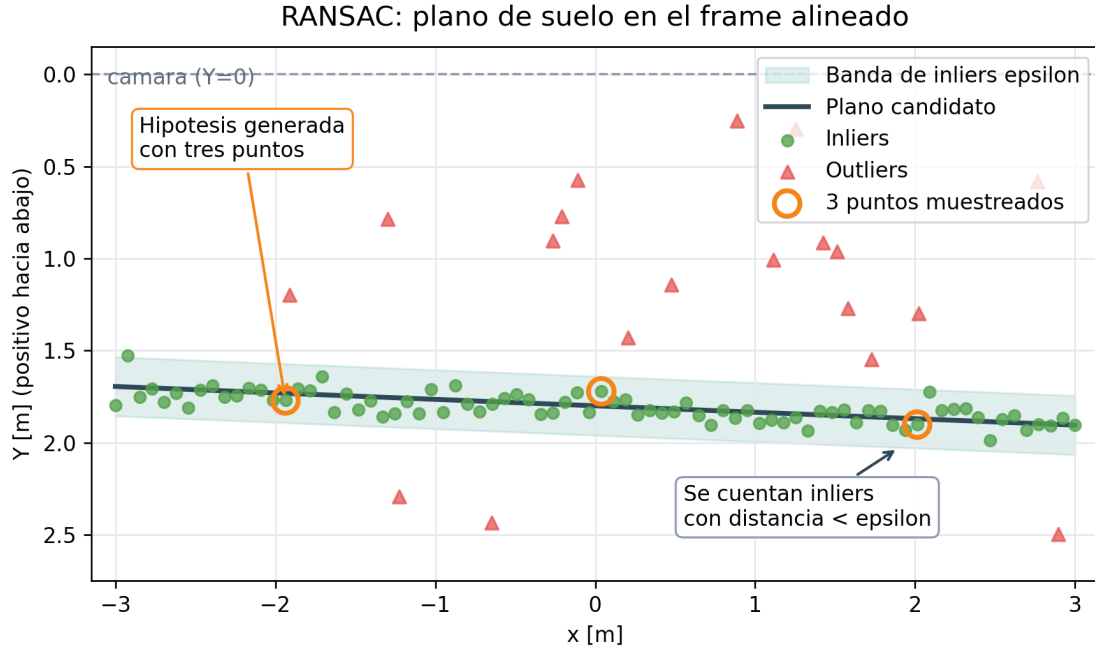


Figura 3.11: Esquema bidimensional de RANSAC para ajuste de planos. Cada hipótesis se genera a partir de tres puntos y luego se cuentan como *inliers* las muestras cuya distancia al plano candidato es menor que la tolerancia ε . Elaboración propia.

donde Y^* se obtiene mediante un histograma de alturas. La calidad se calcula en relación con esa banda:

$$\rho = \frac{|\text{inliers} \cap \mathcal{B}|}{|\mathcal{B}|}. \quad (3.30)$$

Además, como el error axial del D435 crece con la distancia [25], la tolerancia puede variar por punto:

$$\varepsilon_i = \max(\varepsilon, k\|\mathbf{p}_i\|). \quad (3.31)$$

Con este criterio, el plano no se estima a partir de toda la nube por igual. La búsqueda se concentra en la región donde es esperable observar el piso y evalúa la calidad dentro de esa región, incorporando además una tolerancia compatible con el ruido de profundidad.

Estos recursos proporcionan los fundamentos para construir un estimador robusto, pero su orden y sus condiciones de aceptación dependen del método adoptado. El procedimiento utilizado en este trabajo, incluidos la reutilización temporal, los caminos de fallo y la clasificación final, se presenta en la Sección ??.

3.2.11. Clasificación geométrica y región superior excluida

Una vez obtenido el plano, la distancia con signo permite separar los puntos compatibles con el suelo de los obstáculos. Se etiqueta como suelo un punto dentro de la tolerancia o situado por debajo del plano estimado. Entre los restantes, los puntos cuya coordenada satisface $Y < -\Delta_c$ se asignan a una región superior excluida, con $\Delta_c = 0,10$ m en la configuración analizada; los demás se conservan como obstáculos.

Esta región no constituye una detección semántica de techos. Reúne cualquier medición situada más de Δ_c por encima de la cámara y puede contener ramas, carteles u otros elementos elevados. Se mantiene el término **CEILING** únicamente al referirse a la etiqueta del mensaje o

del código. Su descarte y el empleo submuestreado de esos puntos como extremos de espacio libre son decisiones que deben validarse experimentalmente, porque una proyección 2D puede perder información sobre obstáculos elevados.

La aplicación de esta clasificación sobre una escena real se muestra en las Figuras ?? y ?. La primera permite observar el cambio de marco previo a la estimación; la segunda muestra la separación resultante entre suelo, región superior excluida y los puntos conservados como obstáculos.

3.2.12. Realimentación de la altura de cámara

La distancia del origen al plano estimado proporciona una medición de la altura de la cámara. Para impedir que errores aislados modifiquen bruscamente la grilla, esta estimación solo se acepta cuando el plano supera requisitos de calidad, cantidad de inliers e inclinación. La altura aceptada se filtra, se limita el cambio por actualización y los saltos grandes deben repetirse de forma coherente antes de iniciar una transición más rápida.

El valor resultante actualiza el límite vertical superior usado por la codificación espacial. De este modo, las cinco filas mantienen una relación con el espacio comprendido entre la zona inferior y la altura observada de la cámara, aun cuando cambie el usuario o la posición de montaje. Esta realimentación modifica la asignación vertical, pero no altera la matriz final de 5 filas por 10 columnas.

El procedimiento adoptado para validar y realimentar esta altura se presenta en la Sección ??; su evaluación experimental se define en el bloque v1 del capítulo de experimentos.

3.3. Fundamentos de la versión v2: odometría, mapa local y conciencia espacial

La versión v2 parte de la percepción filtrada alcanzada en v1 y agrega memoria espacial para cubrir su principal limitación restante. Para que un obstáculo siga influyendo cuando abandona el campo frontal, las observaciones locales deben transformarse a un marco común, actualizar una grilla de ocupación y evaluarse junto con la velocidad del usuario. En la configuración final, esta etapa se apoya en `imu_filter_madgwick`, `rtabmap_odom`, `local_mapper` y `spatial_awareness`.

3.3.1. Filtro de Madgwick para orientación IMU

La incorporación de Madgwick corresponde a v2, no a v1. En v1 la señal inercial relevante es el acelerómetro, porque permite estimar la vertical local a partir de la gravedad con bajo costo. En v2, en cambio, la odometría necesita una orientación IMU filtrada que combine acelerómetro y giróscopo. El driver de la D435i publica mediciones inerciales, pero la orientación del mensaje IMU no constituye por sí misma una orientación filtrada utilizable por odometría.

El problema es que ninguna de las dos mediciones inerciales resuelve por sí sola la orientación de forma robusta. El giróscopo describe bien los cambios rápidos de orientación, pero su integración acumula deriva. El acelerómetro, en cambio, ofrece una referencia de baja frecuencia asociada a la gravedad, útil para corregir roll y pitch cuando la aceleración dinámica no domina la medición [19]. Esta complementariedad motiva el uso de un filtro que propague la orientación con el giróscopo y la corrija con la dirección de gravedad medida por el acelerómetro.

Para este propósito se usa el filtro de Madgwick [24]. Para mantener la correspondencia con el diagrama original, se adopta en esta sección su notación: S es el marco del sensor, E es el marco de referencia, el sombrero indica un cuaternión unitario y t identifica la muestra actual.

La rama inferior de la Figura 3.12 comienza con la derivada de orientación calculada a partir del gir6scopo:

$${}^S_E \dot{\hat{q}}_{\omega,t} = \frac{1}{2} {}^S_E \hat{q}_{est,t-1} \otimes {}^S \omega_t, \quad (3.32)$$

donde ${}^S \omega_t$ es la velocidad angular expresada en el marco del sensor y ${}^S_E \hat{q}_{est,t-1}$ es la orientación estimada en la muestra anterior. En la rama superior, el aceler6metro normalizado ${}^S \hat{a}_t$ define la direcci3n de gravedad observada. La diferencia respecto de la gravedad predicha por el cuaterni3n anterior se expresa mediante la funci3n objetivo f_g ; su jacobiano J_g permite obtener la direcci3n de correcci3n:

$$\nabla f_g = J_g^T ({}^S_E \hat{q}_{est,t-1}) f_g ({}^S_E \hat{q}_{est,t-1}, {}^S \hat{a}_t). \quad (3.33)$$

El sumador central combina ambas ramas restando el gradiente normalizado, ponderado por la ganancia β :

$${}^S_E \dot{\hat{q}}_{est,t} = {}^S_E \dot{\hat{q}}_{\omega,t} - \beta \frac{\nabla f_g}{\|\nabla f_g\|}. \quad (3.34)$$

Finalmente, el bloque de integraci3n actualiza la orientaci3n, el bloque siguiente la normaliza y los dos retardos z^{-1} del diagrama realimentan el valor correspondiente a la muestra anterior:

$${}^S_E \hat{q}_{est,t} = \text{norm} ({}^S_E \hat{q}_{est,t-1} + {}^S_E \dot{\hat{q}}_{est,t} \Delta t). \quad (3.35)$$

En esta secci3n se presentan 6nicamente las relaciones necesarias para interpretar el diagrama y comprender el uso del filtro dentro del sistema. La derivaci3n completa de la funci3n objetivo, su jacobiano, el descenso de gradiente y las expresiones optimizadas para las implementaciones IMU y MARG puede consultarse en el informe interno original de Madgwick [32].

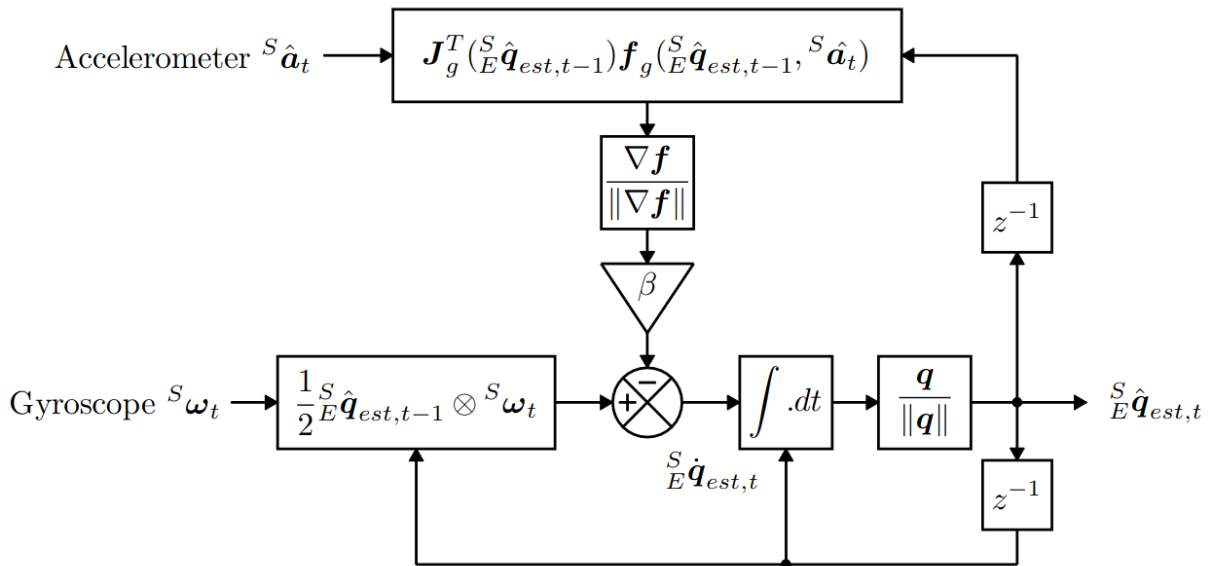


Figura 3.12: Diagrama de bloques del filtro de Madgwick para una implementaci3n IMU sin magnet3metro. La estimaci3n propagada mediante el gir6scopo se corrige por descenso de gradiente a partir de la direcci3n de gravedad observada por el aceler3metro, y luego se integra y normaliza. Fuente: Madgwick [32, Fig. 2].

Sin magnet3metro, la gravedad no hace observable el yaw absoluto. Por eso, en este sistema la correcci3n de la rama superior limita principalmente la deriva de roll y pitch, mientras que la evoluci3n horizontal debe provenir de las correspondencias visuales. La implementaci3n produce

el cuaternión de `/imu/data_filtered`, utilizado para inicializar y regular `rgbd_odometry`; este acoplamiento no constituye una optimización visual-inercial conjunta. La elección de Madgwick responde, además, a su bajo costo de cálculo por muestra, adecuado para ejecución embebida.

3.3.2. Odometría RGB-D con RTAB-Map

La odometría visual estima el movimiento de la cámara a partir de cambios entre fotogramas consecutivos [26]. En v2 se utiliza `rgbd_odometry` de RTAB-Map [23]. El nodo consume imagen RGB, parámetros intrínsecos, profundidad alineada a color y la IMU filtrada. Conceptualmente, detecta rasgos visuales, los asocia entre cuadros y usa la profundidad para convertir sus píxeles en correspondencias métricas 3D. La transformación rígida que mejor explica esas correspondencias produce un incremento de pose. Las asociaciones incorrectas deben rechazarse como valores atípicos; la falta de textura, las oclusiones y el desenfoque por movimiento pueden reducir o interrumpir el seguimiento.

RTAB-Map puede operar como biblioteca de SLAM y mapeo, pero en este sistema se usa su odometría RGB-D como fuente local de pose. La salida se publica como `nav_msgs/Odometry` y se remapea a `nav_odom`. El marco `odom` no representa un sistema global absoluto; es un marco local continuo, con origen definido al inicio de la ejecución y con deriva acumulada a largo plazo. Esta propiedad es suficiente para un mapa local de corto alcance, porque la coherencia requerida abarca segundos y metros recientes, no un mapa permanente de todo lo recorrido previamente.

La secuencia concreta utilizada para documentar esta operación debe obtenerse de datos reales; su incorporación se plantea en la Sección ?? como evidencia del método adoptado, no como fundamento general de la odometría visual.

3.3.3. Marcos de referencia y convención REP-103

ROS define convenciones estándar de unidades y marcos en REP-103 [33]. Sin embargo, los flujos ópticos de cámara utilizan una convención natural para imágenes: x hacia la derecha, y hacia abajo y z hacia adelante. El sistema combina ambas perspectivas: RTAB-Map publica odometría en sus marcos ROS, mientras que el pipeline geométrico interno trabaja con la convención óptica de la cámara.

La transformación entre convenciones no altera la información física; solo expresa la misma pose en ejes compatibles con la grilla local. Para el mapa 2D se conserva la posición horizontal del sensor y el yaw extraído de la orientación. Roll y pitch no se aplican en esta etapa porque la nube ya fue alineada con gravedad en v1. Esta separación evita mezclar la estabilización vertical de la percepción con la acumulación horizontal del mapa.

Para la fuente RTAB-Map, la conversión de la traslación publicada a las coordenadas horizontales internas es:

$$x_{\text{int}} = -y_{\text{ROS}}, \quad z_{\text{int}} = x_{\text{ROS}}. \quad (3.36)$$

La orientación se precompone con el cuaternión $q_{\text{ROS} \rightarrow \text{int}} = (0,5, 0,5, -0,5, 0,5)$, escrito como (w, x, y, z) , y después se proyecta la dirección frontal sobre el plano XZ para conservar únicamente yaw. Cada punto local $\mathbf{p}_l = (x, 0, z)$ se inserta mediante

$$\mathbf{p}_{\text{odom}} = q_{\text{yaw}} \mathbf{p}_l q_{\text{yaw}}^* + (x_{\text{int}}, 0, z_{\text{int}}). \quad (3.37)$$

Estas expresiones fijan tanto el orden del cuaternión como el sentido de la transformación y evitan interpretar un cambio de base como movimiento físico adicional.

La verificación de estas convenciones contra los marcos TF publicados forma parte del método de integración de v2 y se ubica en la Sección ??, donde puede contrastarse con RViz sin convertir una convención específica del sistema en una propiedad general de ROS.

3.3.4. Sincronización temporal de observaciones y odometría

El mapa local combina tres fuentes temporales: nube de obstáculos, extremos de rayos libres y odometría. Las dos nubes derivan del mismo frame de profundidad, por lo que comparten `header.stamp` y pueden sincronizarse con política exacta. La odometría, en cambio, llega a otra frecuencia; por eso se mantiene un buffer y se busca la muestra más reciente no posterior al instante de la observación. Si todavía no existe una muestra anterior, la implementación actual usa la primera muestra disponible del buffer, aunque sea posterior. Solo descarta el frame cuando el buffer está vacío.

Esta aproximación evita detener el mapa durante el arranque, pero no garantiza causalidad ni establece una diferencia temporal máxima. Tampoco interpola entre poses. Por ello, el error espacial asociado a la desincronización debe medirse y esta conducta no se presenta como una sincronización exacta. Una política más estricta podría descartar muestras futuras o interpolar entre poses anterior y posterior, pero no corresponde atribuirle esa conducta al sistema evaluado.

La política temporal concreta y sus casos de arranque se muestran en el diagrama del método de `v2`, en la Sección ??; aquí interesa conservar la distinción conceptual entre sincronización exacta, selección de muestra e interpolación.

3.3.5. Representaciones espaciales locales

Se distingue entre mapas globales, que buscan representar un entorno completo, y mapas locales de corto alcance, centrados en el usuario y limitados a una ventana espacial. Los mapas locales permiten actualizaciones frecuentes con baja latencia, por lo que son adecuados para evasión reactiva [15], [18]. En este trabajo se adopta una grilla de ocupación 2D local en el marco `odom`.

La grilla local se expresa en el plano horizontal. La altura estimada de la cámara respecto del suelo, publicada por el filtro geométrico, se usa para ubicar correctamente el origen de la visualización y mantener consistencia entre la nube 3D, el plano de suelo y la representación 2D.

3.3.6. Grillas de ocupación y log-odds

La grilla de ocupación fue introducida por Elfes [11] como una discretización probabilística del espacio. Cada celda m_i almacena la probabilidad de ocupación condicionada al historial de observaciones $z_{1:t}$. Para actualizar eficientemente se usa la escala de *log-odds*:

$$l_t(m_i) = l_{t-1}(m_i) + \underbrace{\log \frac{P(m_i = 1 | z_t)}{P(m_i = 0 | z_t)}}_{l_{\text{sensor}}(z_t, m_i)} - l_0. \quad (3.38)$$

La ecuación 3.38 convierte la actualización bayesiana recursiva en una suma [12]. Las celdas impactadas por obstáculos reciben un incremento positivo l_{occ} , mientras que las celdas atravesadas por rayos libres reciben un incremento negativo l_{free} . Para estabilidad numérica, l_t se satura en $[l_{\text{mín}}, l_{\text{máx}}]$.

Los incrementos dependen de las probabilidades asignadas por el modelo inverso del sensor:

$$l_{\text{occ}} = \log \frac{p_{\text{occ}}}{1 - p_{\text{occ}}}, \quad l_{\text{free}} = \log \frac{p_{\text{free}}}{1 - p_{\text{free}}}. \quad (3.39)$$

Estas probabilidades expresan cuánto se confía en que una celda esté ocupada después de un impacto o después de ser atravesada por un rayo libre. No son características nominales de

la cámara ni valores fijados por la formulación de Elfes: deben seleccionarse para el sensor, el procesamiento previo y el comportamiento temporal buscado.

La probabilidad puede recuperarse como:

$$P(m_i = 1 \mid z_{1:t}) = 1 - \frac{1}{1 + \exp(l_t(m_i))}. \quad (3.40)$$

Esta forma permite publicar la grilla en `nav_msgs/OccupancyGrid`. La configuración adoptada usa celdas de 0,10 m en una matriz de 100×100 , es decir, una ventana deslizante de 10×10 m centrada en la posición de la cámara. Por lo tanto, sobre una trayectoria recta la ventana abarca aproximadamente 5 m por detrás y 5 m por delante, y la misma extensión hacia ambos laterales. Además, la memoria se restringe a un radio de 5 m: las celdas del cuadrado cuya distancia a la cámara supera ese radio se reinician como desconocidas. La matriz cuadrada define el soporte del mensaje, mientras que el radio define la región circular en la que se conserva información histórica. Como punto de partida previo a la calibración se adoptan $p_{\text{occ}} = 0,9$ y $p_{\text{free}} = 0,3$, que mediante la Ecuación 3.39 producen $l_{\text{occ}} = 2,197$ y $l_{\text{free}} = -0,847$. Estos valores iniciales hacen que una detección ocupada tenga mayor peso que una observación libre aislada, pero no representan una caracterización experimental de la D435i. Su calibración se define en la Sección ???. Se emplea además saturación en $[-5, 5]$.

El valor de log-odds es la representación interna del mapeador; antes de publicar, cada celda debe convertirse al campo entero `data[i]` de `nav_msgs/OccupancyGrid`. Este mensaje reserva -1 para una celda desconocida y usa el intervalo $0-100$ para expresar la probabilidad porcentual de ocupación. En esta implementación, una celda nunca observada conserva exactamente $l = 0$ y se publica como -1 . Para una celda que ya recibió evidencia se recupera P mediante la Ecuación 3.40 y se publica

$$\text{data}[i] = \text{round}(100P). \quad (3.41)$$

Así, por ejemplo, $P = 0,90$ se publica como 90. Estos enteros no son distancias ni intensidades hápticas: representan el contenido de una celda del mapa. Los valores menores que 50 favorecen la hipótesis de espacio libre y los mayores que 50 favorecen la de ocupación; el umbral operativo usado posteriormente para clasificar una celda se define por separado.

La extensión de la ventana no implica que exista una medición en toda esa superficie. La cámara aporta observaciones dentro de su campo de visión y alcance; la zona posterior y las zonas laterales conservan únicamente información observada previamente mientras permanezcan dentro del radio local. Las regiones todavía no observadas continúan en estado desconocido.

Por ejemplo, una observación ocupada lleva una celda desconocida a $l = 2,197$, equivalente a $P \simeq 0,90$ y a `data[i] = 90`. Una observación libre posterior la reduce a $l = 1,350$, equivalente a $P \simeq 0,79$ y `data[i] = 79`. Las observaciones libres siguientes producen sucesivamente 62 y 41. Se necesitan, por lo tanto, observaciones contradictorias repetidas para invertir la evidencia. Este ejemplo muestra tanto la memoria temporal interna como su codificación en el mensaje publicado.

La Figura 3.13 representa la secuencia numérica anterior. En cada punto se muestran por separado el log-odds del eje vertical, la probabilidad recuperada y el valor de `OccupancyGrid.data[i]`. El cruce por $l = 0$ permite identificar cuándo la evidencia acumulada deja de favorecer ocupación y pasa a favorecer espacio libre.

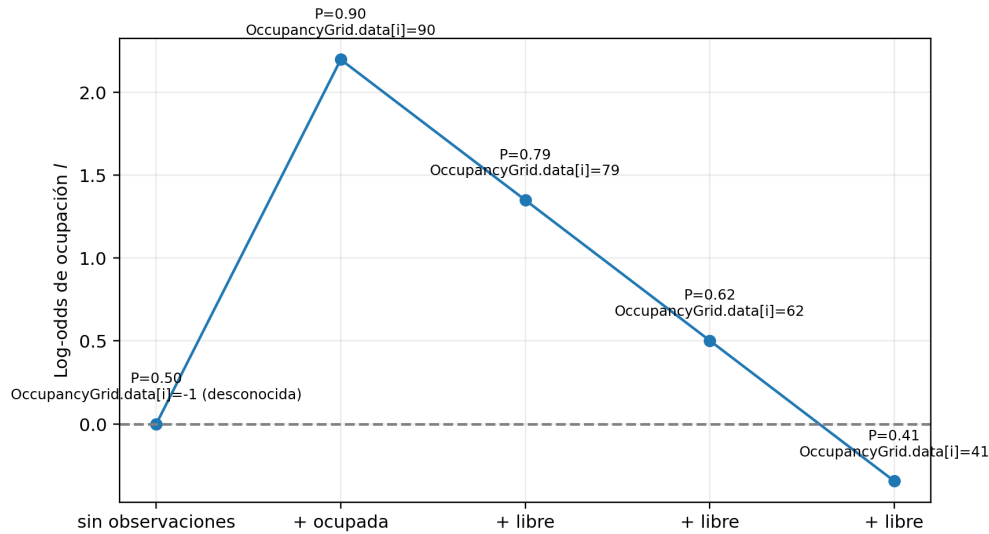


Figura 3.13: Evolución del log-odds de una celda inicialmente desconocida y conversión de cada estado al valor publicado en `OccupancyGrid.data[i]`, utilizando la configuración inicial previa a la calibración.

3.3.7. Ray casting con Bresenham

Además de marcar ocupada la celda del obstáculo, el mapa debe marcar como libre el espacio atravesado por el rayo de medición. El segmento continuo medido por la cámara debe convertirse, por lo tanto, en una sucesión de celdas de la grilla. Para ello se usa el algoritmo de Bresenham [16]. Dado un origen (x_0, z_0) y un destino (x_1, z_1) en coordenadas enteras de celda, el algoritmo avanza sobre el eje dominante y decide cuándo incrementar el eje subordinado mediante una variable de error acumulado. El resultado es una aproximación discreta del segmento formada por celdas contiguas; no es el conjunto geométrico de todas las celdas que la recta llega a tocar. Esta construcción evita calcular intersecciones en punto flotante.

En este sistema se aplican dos tipos de rayos. Los rayos con obstáculo marcan libres las celdas intermedias y ocupada la celda final. Los rayos asociados a suelo, región superior excluida o mediciones sin obstáculo dentro del rango marcan libre también el extremo final. Para no borrar evidencia acumulada de forma excesiva, una celda recibe evidencia libre como máximo una vez por frame.

La Figura 3.14 muestra la discretización espacial de ambos casos. La línea azul une, en orden, los centros de las celdas seleccionadas por Bresenham desde la cámara hasta el destino; por eso no debe interpretarse como la intersección geométrica exacta entre una recta y la grilla. Las celdas verdes reciben evidencia libre. Si el extremo corresponde a un obstáculo, la última celda se reserva para evidencia ocupada (rojo). Si la medición no contiene un obstáculo dentro del rango, el recorrido se extiende hasta el límite medible representado y su extremo también recibe evidencia libre.

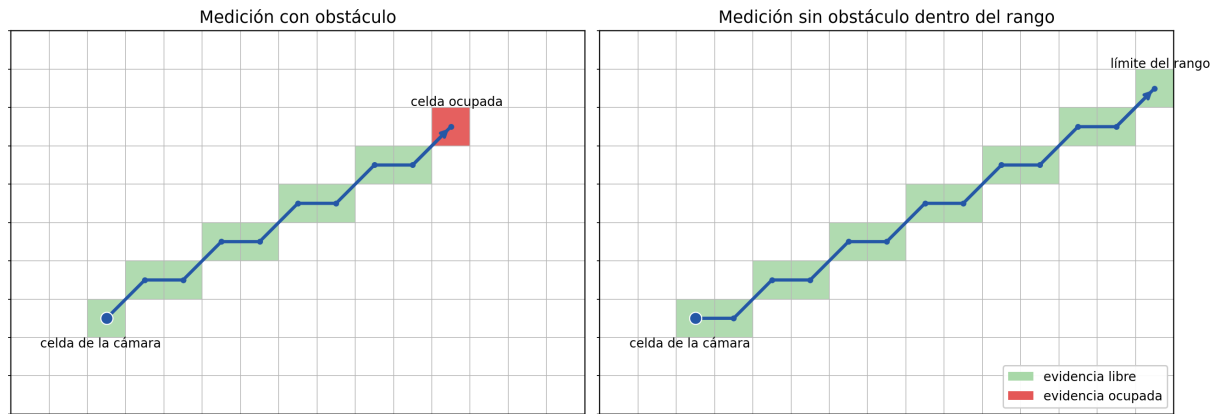


Figura 3.14: Recorrido discreto de Bresenham sobre la grilla: las celdas intermedias reciben evidencia libre; ante un obstáculo el extremo es ocupado y, en caso contrario, la evidencia libre se extiende hasta el límite del rango representado.

3.3.8. Ventana local deslizante y olvido espacial

La memoria espacial se mantiene en una ventana finita. Cuando la odometría indica desplazamiento del sensor, la grilla se desplaza una cantidad entera de celdas y las celdas nuevas se reinician con prior neutro. Para evitar pérdida de resolución a bajas velocidades, los desplazamientos sub-celda se acumulan hasta superar el tamaño de celda.

Además del corrimiento de ventana, se emplea un radio de olvido local: las celdas demasiado alejadas del sensor vuelven al prior. Este mecanismo impide que observaciones viejas dominen indefinidamente y mantiene el mapa enfocado en la región útil para navegación inmediata.

La ventana está centrada alrededor de la posición estimada del usuario, pero sus ejes permanecen alineados con *odom*: no gira con cada cambio de yaw. La traslación desplaza los contenidos por una cantidad entera de celdas y un acumulador conserva las fracciones menores que 0,10 m hasta completar una celda. Las posiciones de los obstáculos se rotan y trasladan al marco común antes de actualizar la grilla.

El corrimiento entero, la acumulación subcelda y la conservación de los ejes de *odom* son decisiones del método de actualización adoptado; su diagrama se presenta en la Sección ??.

3.3.9. Filtrado espacial y temporal

El filtrado espacial aparece en dos niveles. Primero, la reducción a *DepthGrid* resume múltiples muestras en celdas. Segundo, la grilla de ocupación acumula evidencia por celda en el plano local. El filtrado temporal surge de la actualización log-odds: observaciones persistentes refuerzan ocupación y mediciones libres repetidas la reducen. Este comportamiento suaviza detecciones esporádicas, aunque requiere saturación y olvido para evitar que el mapa se vuelva rígido frente a cambios reales. En la implementación actual no se aplica un decaimiento explícito por edad de celda: una ocupación pierde peso cuando es contradicha por rayos libres, cuando queda fuera del radio local o cuando sale de la ventana deslizante.

3.3.10. Conciencia espacial y tiempo hasta colisión

La capa de conciencia espacial evalúa riesgos fuera del campo frontal ya comunicado por la grilla háptica. Primero selecciona celdas cuyo valor de ocupación alcanza 65 y agrupa componentes con vecindad de ocho conexiones. Solo conserva componentes de al menos tres

celdas. Esta regla reduce activaciones aisladas, pero es una heurística: también puede omitir obstáculos delgados o representados por pocas celdas y, por ello, requiere validación.

Para cada celda candidata se calcula su posición relativa respecto de la pose del usuario y se descarta si cae dentro de la semiamplitud frontal observable de 45° , región ya atendida por la percepción frontal. Luego se evalúa si intersecta una zona compatible con colisión, orientada según el vector de velocidad y no necesariamente según el frente observable. Su semiancho considera el radio corporal y la mitad de la diagonal de una celda de 0,10 m:

$$r_c = 0,20 + \frac{0,10}{\sqrt{2}} \simeq 0,27 \text{ m.} \quad (3.42)$$

Esta tolerancia incluye celdas cuyo centro queda ligeramente fuera del radio corporal, pero cuya extensión todavía puede intersectar la trayectoria. Finalmente se exige velocidad de cierre suficiente. Solo se analizan distancias de 0,20 a 1,00 m y velocidades de movimiento de al menos 0,10 m/s. La velocidad de cierre se calcula como

$$v_c = \max \left(0, \mathbf{v} \cdot \frac{\mathbf{p}_o - \mathbf{p}_u}{\|\mathbf{p}_o - \mathbf{p}_u\|} \right), \quad (3.43)$$

donde $\mathbf{p}_o$ y $\mathbf{p}_u$ son las posiciones del obstáculo y del usuario. Para que el horizonte represente la llegada del obstáculo al contorno corporal y no al centro de la pose, la separación empleada es

$$d = \max(0, \|\mathbf{p}_o - \mathbf{p}_u\| - r_{\text{cuerpo}}), \quad r_{\text{cuerpo}} = 0,20 \text{ m.} \quad (3.44)$$

El tiempo hasta colisión se define entonces como:

$$t_{tc} = \frac{d}{v_c}, \quad (3.45)$$

donde d es la separación entre el contorno corporal y el obstáculo, mientras que v_c es la componente de velocidad dirigida hacia él. El modelo supone obstáculo estacionario y velocidad del usuario constante durante el horizonte analizado; no predice el movimiento de terceros. La ecuación 3.45 prioriza urgencia dinámica, no solo proximidad geométrica. La intensidad asociada puede escalarse linealmente entre un tiempo mínimo y un horizonte máximo:

$$I_{\text{riesgo}} = 100 \text{ sat}_{[0,1]} \left(\frac{t_{\text{máx}} - t_{tc}}{t_{\text{máx}} - t_{\text{mín}}} \right). \quad (3.46)$$

La configuración usa $t_{\text{mín}} = 0,25$ s y $t_{\text{máx}} = 2,00$ s. Los riesgos se clasifican en izquierda, derecha y posterior; la región posterior abarca 45° a cada lado de la dirección opuesta al frente. En cada región se conserva el candidato con mayor intensidad y, ante empate, menor TTC. Una histéresis de 0,03 m/s reduce conmutaciones alrededor del umbral de velocidad de cierre. La evaluación también exige que mapa y odometría tengan una antigüedad no mayor que 0,40 s.

La velocidad se filtra con $\alpha = 0,25$ y se limita a 3 m/s. El sistema compara la velocidad informada por la odometría con la derivada de poses consecutivas y usa una tolerancia de consistencia de 0,35 m/s para seleccionar una fuente plausible. Esta selección debe explicitarse porque el TTC depende linealmente de la velocidad elegida.

La Figura 3.15 reúne estas relaciones en una vista superior. El vector $\mathbf{v}$ determina el eje de la zona compatible con colisión; v_c es su proyección sobre la dirección del obstáculo. La distancia d se mide desde el contorno corporal y no desde el centro de la pose. Los valores numéricos constituyen un ejemplo geométrico reproducible, no un resultado experimental.

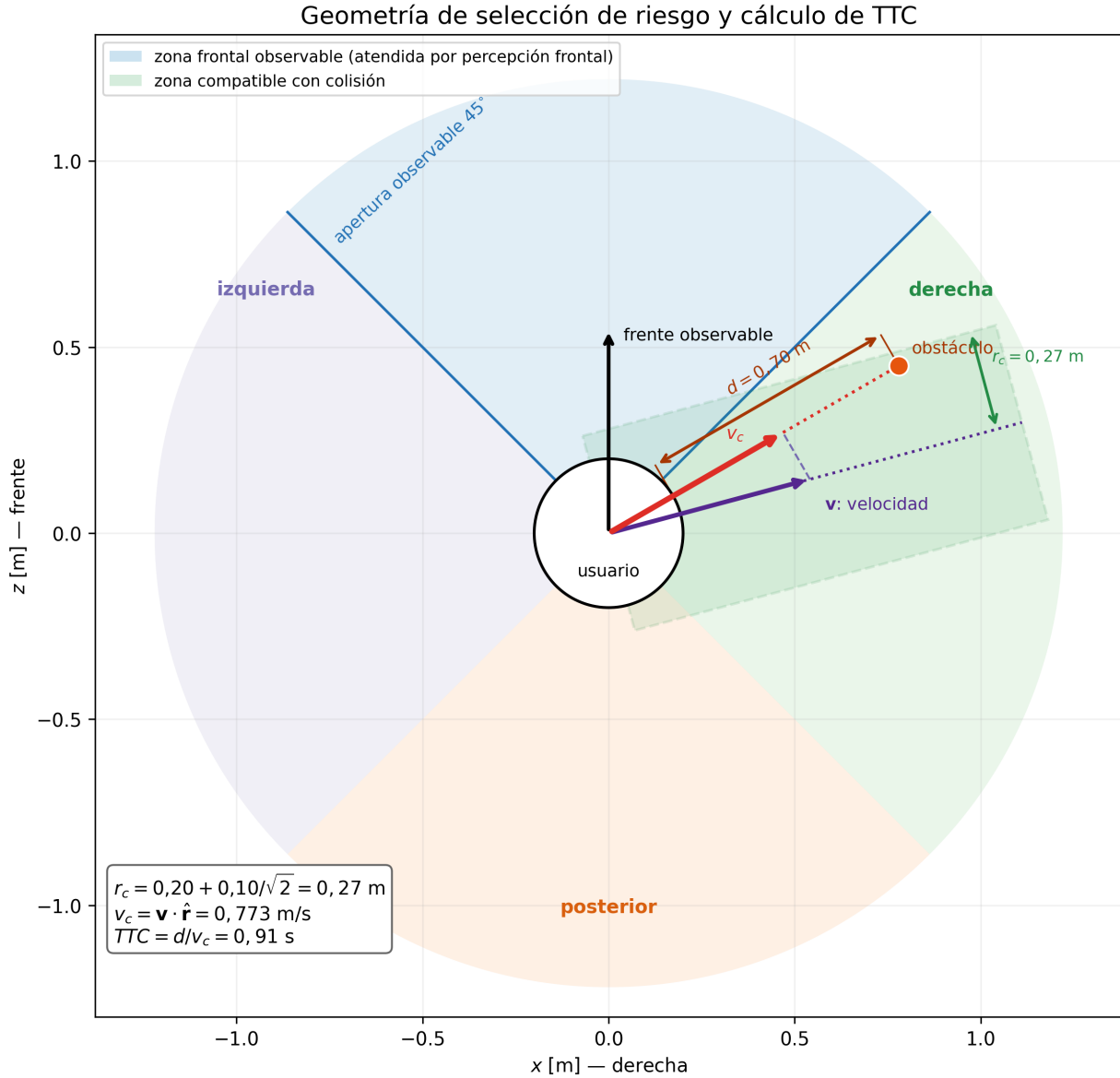


Figura 3.15: Geometría empleada para seleccionar un riesgo contextual y calcular el tiempo hasta colisión. La zona frontal observable queda a cargo de la percepción frontal; la evaluación contextual considera las regiones laterales y posterior.

3.3.11. Fusión háptica con riesgo contextual

En v2, la salida háptica ya no representa únicamente la percepción frontal instantánea. La grilla frontal sigue codificando distancia a obstáculos en el campo visual, mientras que la conciencia espacial aporta señales semánticas de riesgo lateral y posterior. La fusión se realiza por máximo de intensidad: si una celda ya está activa por percepción frontal, una señal contextual solo la aumenta si su intensidad es mayor.

La implementación aplica el riesgo izquierdo a las columnas del borde izquierdo, el derecho al borde derecho y el posterior a las columnas centrales; en la configuración por defecto se usan una, una y dos columnas, respectivamente. La intensidad contextual se conserva durante un máximo de 0,75 s sin una actualización válida.

Esta regla conserva el mayor valor numérico sin promediar señales, pero no demuestra por sí sola que se conserve el riesgo físicamente más urgente: la intensidad frontal deriva de distancia y

la contextual de TTC. Su comparación presupone que ambas escalas 0–100 fueron calibradas de manera compatible. Asimismo, la codificación posterior en columnas centrales es una convención que debe verificarse con usuarios y con el dispositivo físico del proyecto.

La obtención de las tres matrices reproducibles corresponde a la validación funcional de v2 y se especifica en el bloque experimental de memoria espacial y riesgo contextual. Aquí se mantiene únicamente la definición de la operación de fusión y la advertencia sobre la compatibilidad de escalas.

3.3.12. Métricas de evaluación para percepción y mapeo

Las métricas de evaluación siguen el orden incremental v0–v1–v2. En v0 se evalúan latencia *end-to-end*, tasa de publicación, coherencia entre distancia e intensidad y costo de transmisión UART. En v1 se agregan error geométrico respecto del plano de suelo, estabilidad de la altura de cámara, tasa de falsos positivos por suelo o techo y costo computacional del filtrado geométrico. En v2 se evalúan precisión y recall de la grilla local, estabilidad temporal por celda, vigencia de odometría, cobertura de riesgo lateral y posterior, y consumo de CPU y memoria. Esta organización conecta la teoría con los bloques experimentales y evita mezclar métricas de capacidades que aparecen en versiones distintas.

Para evitar interpretaciones ambiguas, la latencia *end-to-end* debe definirse entre el sello temporal de adquisición y la publicación de la grilla háptica, y la tasa como mensajes válidos por segundo durante una ventana estable. Para una clasificación binaria de ocupación:

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \frac{\text{precision recall}}{\text{precision} + \text{recall}}. \quad (3.47)$$

La estabilidad temporal debe medirse sobre celdas correspondientes al mismo lugar en *odom*, no sobre el mismo índice antes y después de desplazar la ventana. Las condiciones, referencias y procedimientos concretos de medición se establecen en el capítulo experimental; este apartado solo fija el significado de cada magnitud.

Capítulo 4

Arquitectura del sistema

4.1. Arquitectura general

El sistema se compone de un conjunto de nodos ROS 2 que se ejecutan sobre una Raspberry Pi 5, orquestados por el archivo de lanzamiento `nav.launch.py` del paquete `nav_bringup`. Todos los nodos de procesamiento quedan contenidos en la misma unidad de cómputo y se comunican exclusivamente a través del middleware DDS (`rmw_cyclonedds`). La frontera de la implementación física se ubica en la transmisión UART: el sistema genera y serializa los comandos de intensidad, pero no incluye el receptor, la electrónica de potencia ni los actuadores. Distintos nodos de visualización del paquete (`output_viewer`) pueden ejecutarse opcionalmente en una PC separada, suscribiéndose a los mismos tópicos a través de la red DDS. La topología completa del sistema en su versión final integra percepción instantánea, conciencia espacial y mapeo local, tal como se muestra en la [Figure 4.3](#).

Para facilitar la trazabilidad entre la descripción de la arquitectura, las decisiones metodológicas y el código fuente, véase el [Anexo 11](#), que incluye un mapeo `nodo → tópicos → archivos fuente`.

4.2. Evolución arquitectónica por versiones (v0, v1, v2)

Esta sección se limita a mostrar cómo cambia la topología entre las tres versiones incrementales definidas en la [Sección 1.3.3](#). La implementación conserva los modos `raw` y `filtered`, seleccionados mediante `pipeline_mode`, para ejecutar y comparar las configuraciones alcanzadas en v0 y v1.

v0: modo *raw*. La versión inicial utiliza exclusivamente la percepción instantánea: imagen de profundidad de la cámara, conectada al pipeline mediante remapeos definidos en `nav_bringup`, reducción a `DepthGrid` en `depth_grid_encoder`, fusión trivial (`passthrough`), visualización de la salida y serialización para su transmisión UART. Esta versión prioriza tiempo de respuesta y simplicidad de integración. [Figura 4.1](#).

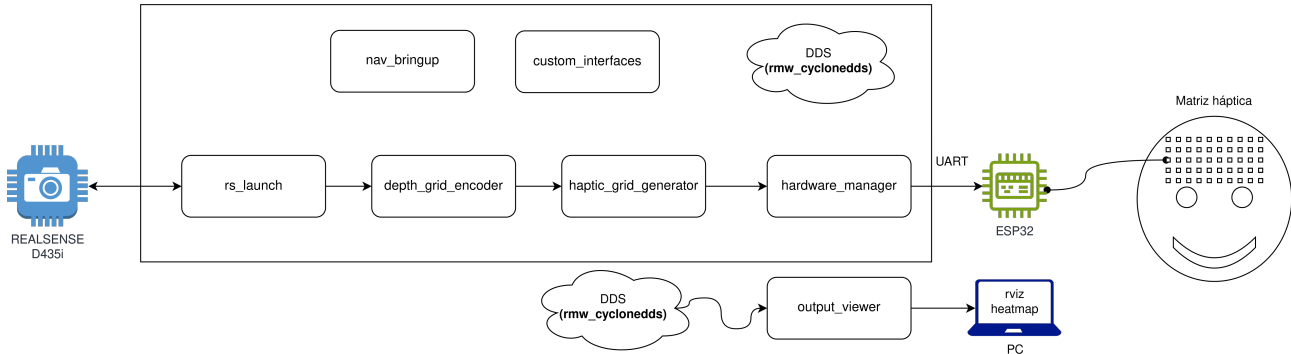


Figura 4.1: Arquitectura v0.

v1: modo *filtered*. Sobre v0 se incorpora `depth_obstacle_filter` para suprimir suelo y techo, alineando la nube a gravedad y conservando en la salida únicamente obstáculos relevantes para la marcha. Esta versión requirió varias iteraciones y optimizaciones respecto de su primera implementación para alcanzar una operación en tiempo real. La arquitectura resultante se muestra en la Figura 4.2.

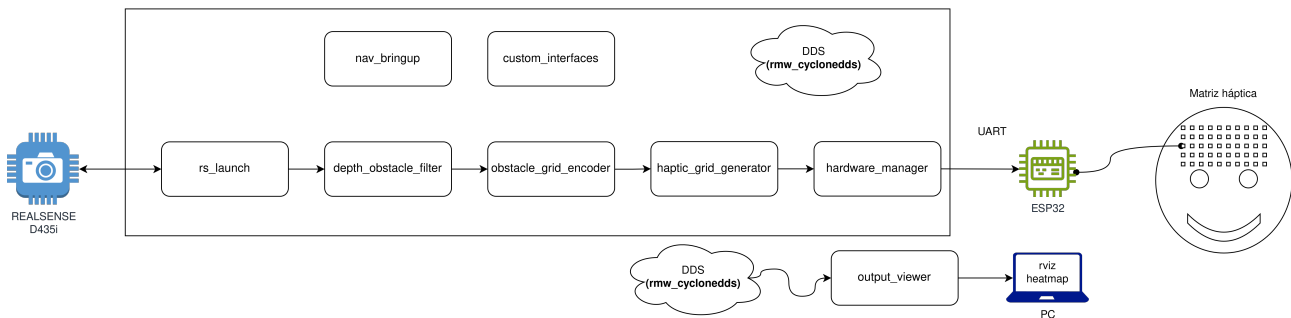


Figura 4.2: Arquitectura v1.

v2: mapeo local + conciencia espacial. La tercera versión agrega memoria espacial mediante `local_mapper` y una capa de riesgo contextual con `spatial_awareness`. Esta capa complementa la percepción frontal instantánea con alertas de ocupación lateral o trasera, aprovechando información acumulada en el mapa local y estado de movimiento del usuario. Figura 4.3.

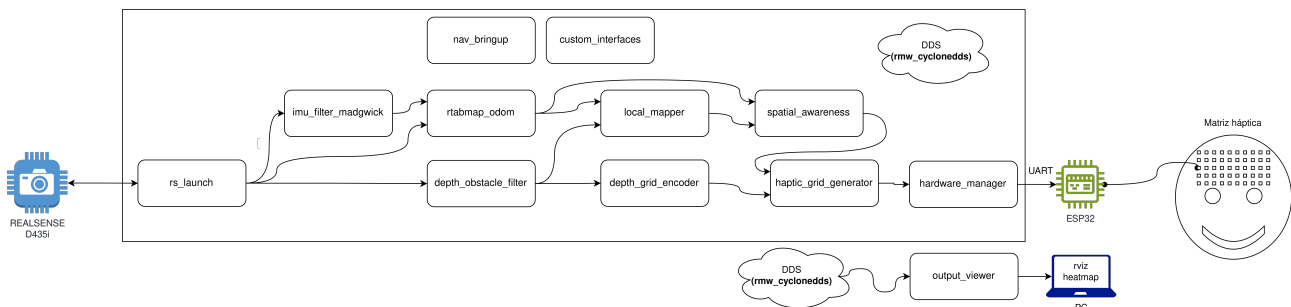


Figura 4.3: Arquitectura v2.

4.3. Paquetes de soporte e infraestructura

El paquete `custom_interfaces` concentra los tipos de mensajes específicos del sistema. En la cadena de percepción frontal, `DepthGrid` modela una grilla compacta de celdas y `DepthCellStats` encapsula la estadística de cada celda (media, mínimo, máximo y conteo de puntos válidos). En la cadena de actuación, `HapticGrid` representa la salida final de intensidades hápticas con dos arreglos explícitos: `intensities` y `active`. Para la conciencia espacial, `DirectionalRisk` describe el riesgo por dirección (activación, intensidad, distancia, velocidad de cierre y tiempo hasta colisión) y `SpatialAwareness` agrupa los tres canales `left`, `right` y `rear` junto con una bandera de validez global. Esta separación reduce acoplamiento entre nodos y evita codificar semántica de riesgo como sobrecapa de un mensaje de percepción.

El paquete `nav_bringup` centraliza la configuración de lanzamiento del sistema en `nav.launch.py`. Este archivo expone parámetros para habilitar o deshabilitar componentes y modos de ejecución, entre ellos: selección de pipeline de percepción (`raw/filtered/none`), activación de `local_mapper` y `spatial_awareness`, publicación en hardware y herramientas de visualización y depuración. Estas opciones determinan qué nodos se inician y con qué parámetros, mientras que la lógica de procesamiento permanece en los paquetes funcionales.

4.4. Adquisición y normalización de tópicos

El driver de la cámara, lanzado mediante `rs_launch.py` del paquete `realsense2_camera`, publica los flujos del sensor bajo el espacio de nombres `/camera/camera/`. En lo que sigue, para evitar repetición, ese prefijo se abrevia como `RS/`. En esta interfaz conviene distinguir entre nombre de tópico y tipo de mensaje: por ejemplo, `RS/depth/image_rect_raw`, `RS/color/image_raw` y `RS/aligned_depth_to_color/image_raw` son tópicos de imagen, todos transportados con tipo `sensor_msgs/Image`, mientras que `RS/depth/camera_info` y `RS/color/camera_info` son tópicos distintos que publican parámetros intrínsecos con tipo `sensor_msgs/CameraInfo`. De forma análoga, las mediciones inerciales llegan por tópicos específicos del driver y utilizan `sensor_msgs/Imu` como tipo de mensaje.

El launcher `nav.launch.py` define remapeos explícitos para cada consumidor, desacoplando la lógica de procesamiento de los nombres publicados por el driver y preservando trazabilidad sobre qué flujo alimenta a cada nodo. Las correspondencias de referencia se indican en la Table 4.1.

Tópico de origen (driver)	Tópico normalizado
<code>RS/depth/image_rect_raw</code>	<code>/sensors/depth/image</code>
<code>RS/depth/camera_info</code>	<code>/sensors/depth/camera_info</code>
<code>RS/color/image_raw</code>	<code>/sensors/color/image</code>
<code>RS/color/camera_info</code>	<code>/sensors/color/camera_info</code>
<code>RS/accel/sample</code>	<code>/sensors/imu/accel</code>
<code>RS/gyro/sample</code>	<code>/sensors/imu/gyro</code>

Tabla 4.1: Correspondencias de tópicos implementadas mediante remapeos en `nav.launch.py`. En esta tabla, `RS/` abrevia el prefijo `/camera/camera/`.

Además de estos remapeos, el código actual consume otros flujos del driver de forma directa. `depth_obstacle_filter` utiliza `RS/depth/image_rect_raw`, `RS/depth/camera_info` y `RS/accel/sample`; el filtro de Madgwick (paquete `imu_filter_madgwick`) consume `RS/imu` para

producir `/imu/data_filtered`; y `rtabmap_odom` utiliza `RS/color/image_raw`, `RS/color/camera_info`, `RS/aligned_depth_to_color/image_raw` e `/imu/data_filtered`. La Table 4.1 resume únicamente los remapeos explícitos usados como interfaz de normalización, no la totalidad de tópicos de cámara presentes en ejecución.

4.5. Versión v0: solución frontal directa

El nodo `depth_to_matrix` (paquete `depth_grid_encoder`) suscribe `depth/image`, remapeado desde `RS/depth/image_rect_raw` en `nav.launch.py`, y reduce la imagen de profundidad densa a una grilla compacta de $R \times C$ celdas (por defecto, 5 filas $\times$ 10 columnas) mediante partición en regiones de interés y cálculo de estadísticas por celda. El resultado se publica en `/perception/depth_grid` como `custom_interfaces/msg/DepthGrid`. Esta versión establece la referencia funcional de menor latencia sobre la cual se evalúan las mejoras posteriores.

La Figura 4.4 explicita el contrato de datos de esta ruta. Cada flecha se lee en tres niveles: nombre de tópico, tipo de mensaje y contenido relevante. La entrada conserva la profundidad del sensor como una imagen `16UC1` expresada en milímetros; el codificador la convierte a una `DepthGrid` de 5×10 celdas cuyas estadísticas se expresan en metros. A partir del mínimo de cada celda, `haptic_grid_generator` produce la `HapticGrid` de intensidades lógicas que recibe `hardware_manager`. De esta forma, el diagrama distingue la representación geométrica de la orden háptica y evita atribuir al gestor de hardware el cálculo de intensidad.

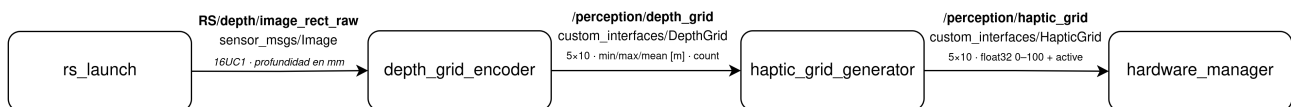


Figura 4.4: Mensajes funcionales de v0. La imagen de profundidad se reduce a `DepthGrid`, se transforma en una matriz lógica `HapticGrid` y se entrega a la interfaz de transmisión.

4.6. Versión v1: refinamiento de la percepción frontal

El paquete `depth_obstacle_filter` encapsula la etapa de percepción profunda. Su nodo `depth_obstacle_filter_node` suscribe la imagen de profundidad, `depth/camera_info` y el acelerómetro IMU del driver (`RS/accel/sample`, remapeado a `imu`), y aplica la siguiente secuencia: retroproyección pinhole a nube de puntos 3D (`DepthProjector`), alineación gravitacional a partir del vector de aceleración filtrado internamente, y detección del plano del suelo por RANSAC jerárquico (`GroundEstimator`) con clasificación de puntos en suelo, obstáculo y techo. El nodo publica `/depth_obstacle_filter/obstacle_cloud` como salida principal, y cuando se habilita la interfaz hacia mapeo local también publica `/depth_obstacle_filter/free_endpoints`. De forma adicional emite `/depth_obstacle_filter/camera_height` (`std_msgs/Float32`) para informar la altura estimada de cámara respecto del suelo. En TF publica la transformación dinámica `gravity_aligned_frame` $\rightarrow$ `camera_depth_optical_frame`. La nube de obstáculos es consumida por `obstacle_grid_encoder`, del paquete `depth_grid_encoder`, que la codifica angularmente y publica `/perception/depth_grid`. De este modo, v0 y v1 ofrecen el mismo contrato `DepthGrid` a las etapas posteriores, aunque lo construyen a partir de fuentes diferentes.

La Figura 4.5 muestra ese reemplazo sin presentar v1 como una cadena independiente. `depth_obstacle_filter` necesita tres entradas del sensor: profundidad rectificada, parámetros intrínsecos y aceleración lineal. Sus salidas funcionales hacia el codificador son la nube de obstáculos, expresada en metros en el marco alineado con gravedad, y la altura filtrada de

cámara. La primera determina qué puntos se agregan; la segunda actualiza el límite vertical usado para distribuirlos entre las cinco filas. El resultado vuelve a ser el mismo tipo `DepthGrid` de `v0`, por lo que la codificación háptica y la interfaz de salida no necesitan cambiar.

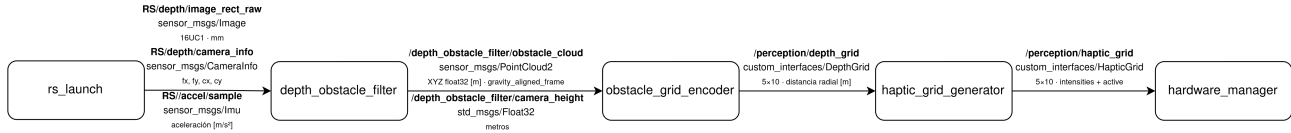


Figura 4.5: Mensajes funcionales de `v1`. La imagen, los intrínsecos y el acelerómetro permiten obtener una nube de obstáculos alineada y codificarla conservando el contrato `DepthGrid`–`HapticGrid` de `v0`.

Esta decisión separa responsabilidades dentro de la cadena: la orientación completa estimada por `imu_filter_madgwick` se reserva para la odometría visual-inercial, mientras que `depth_obstacle_filter` requiere únicamente la dirección de gravedad para estimar roll y pitch. En este contexto, el yaw no modifica la segmentación suelo-obstáculo, por lo que consumir acelerómetro directo reduce dependencias entre nodos, evita latencia adicional en la etapa de percepción y mantiene operativo el filtro geométrico aun cuando el flujo de odometría no esté inicializado. Los detalles de cada etapa se describen en las Secciones ?? a ??.

4.7. Versión `v2`: memoria espacial y protección contextual

La versión `v2` mantiene en la rama inferior de la Figura 4.6 la percepción frontal de `v1` y agrega dos recorridos auxiliares. El primero obtiene `nav_odom` mediante odometría RGB-D: RTAB-Map consume color, sus intrínsecos, imagen de profundidad registrada en el marco óptico de la cámara RGB y la orientación publicada por `imu_filter_madgwick`. El segundo combina esa odometría con las nubes de evidencia ocupada y libre para construir `OccupancyGrid`. A partir del mapa y del estado de movimiento, `spatial_awareness` publica el riesgo contextual que vuelve a `haptic_grid_generator`. Por lo tanto, la rama superior no reemplaza la percepción instantánea: aporta una segunda entrada que se fusiona antes de producir la única `HapticGrid` de salida.

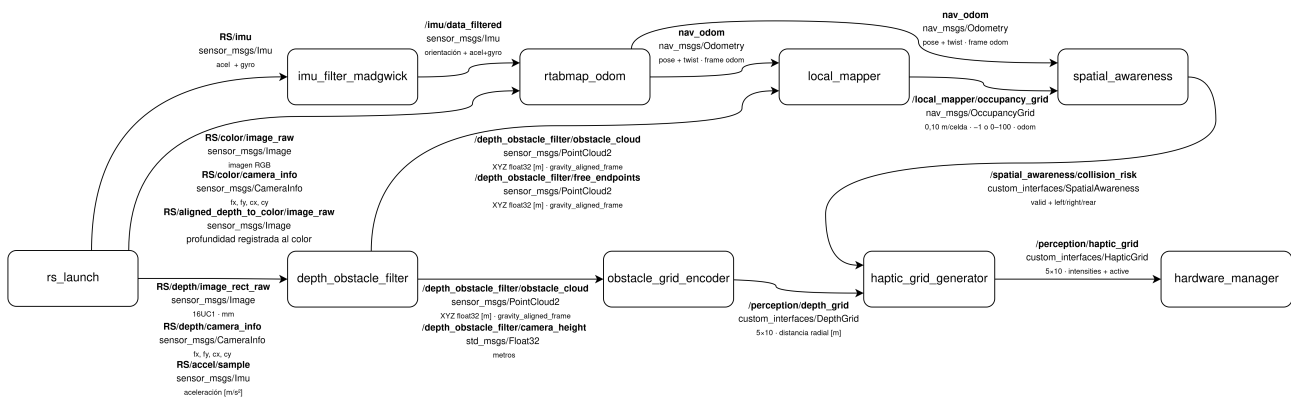


Figura 4.6: Mensajes funcionales de `v2`. La ruta frontal se complementa con odometría, memoria local y riesgo contextual, que se fusiona antes de publicar la matriz háptica final. Se muestran únicamente los mensajes que intervienen en el procesamiento funcional principal.

El nodo `occupancy_mapper_node` (paquete `local_mapper`) es exclusivamente responsable de la acumulación del mapa de ocupación. Suscribe `/depth_obstacle_filter/obstacle_cloud` y `/depth_obstacle_filter/free_endpoints` mediante `ExactTimeSynchronizer`, mientras que la odometría se recibe por `nav_odom` y se mantiene en un buffer temporal para asociar la muestra más reciente no posterior al `stamp` de cada frame de profundidad. Además consume `/depth_obstacle_filter/camera_height` para fijar la componente vertical del origen publicado del mapa. Con estas entradas ejecuta actualización en log-odds con ray casting de Bresenham sobre una rejilla local deslizante (`OccupancyMapper`) y publica `nav_msgs/OccupancyGrid` en `/local_mapper/occupancy_grid` con frame `odom`. También publica la TF dinámica `odom → gravity_aligned_frame` para mantener coherencia de marcos entre mapeo y depuración.

4.8. Conciencia espacial

El nodo `spatial_awareness_node` (paquete `spatial_awareness`) suscribe la grilla local `/local_mapper/occupancy_grid`, la odometría `nav_odom` y el estado de apertura angular (`/perception/depth_grid/aperture_state_deg`). A partir de estas entradas evalúa riesgo fuera del FOV frontal en tres sectores: izquierdo, derecho y posterior. El resultado se publica en `/spatial_awareness/collision_risk` con tipo `custom_interfaces/msg/SpatialAwareness`, que contiene un canal `DirectionalRisk` por sector y un indicador `valid` para señalar vigencia temporal de mapa y odometría.

4.9. Fusión háptica

El nodo `haptic_grid_generator_node` (paquete `haptic_grid_generator`) recibe la percepción instantánea (`/perception/depth_grid`) y el riesgo contextual (`/spatial_awareness/collision_risk`). Para cada celda válida transforma la distancia mínima en una intensidad lógica entre 0 y 100 mediante la ley lineal invertida y saturada de la Ecuación (3.8). Cuando el mensaje de conciencia espacial está vigente y marcado como válido, se superpone la intensidad de riesgo sobre columnas laterales y posteriores de la matriz háptica. La salida consolidada se publica en `/perception/haptic_grid` con tipo `custom_interfaces/msg/HapticGrid`.

4.10. Interfaz lógica de salida y transmisión UART

El nodo `grid_to_motor` (paquete `hardware_manager`) suscribe `/perception/haptic_grid`; por lo tanto, recibe comandos de intensidad ya calculados y no procesa distancias. Para cada celda activa redondea y satura la intensidad al intervalo $[0, 100]$, la escala según `duty_max` mediante la Ecuación (3.9) y serializa los valores resultantes. Las celdas inactivas se transmiten con valor cero. El formato de trama UART es ASCII: $M \ D_0 \ D_1 \ \dots \ D_{N-1} \backslash n$ a 115200bps en modo 8N1 sin control de flujo. Un mecanismo de *throttling* basado en marcas de tiempo descarta mensajes si el intervalo desde el último envío es inferior al período configurado (`send_period_ms`), evitando saturar el buffer UART. La implementación y validación del receptor, de la electrónica de potencia y de los motores vibrotáctiles quedan fuera del alcance de esta tesis. La eficiencia del protocolo y sus alternativas para matrices de mayor densidad se discuten en ??.

4.11. Visualización y depuración

El nodo `depth_grid_heatmap` (paquete `output_viewer`), implementado en Python, puede ejecutarse opcionalmente en una PC conectada a la misma red DDS. Genera una visualización en mapa de calor de `/perception/haptic_grid` y puede superponer el estado de `/spatial_awareness/collision_risk` para inspección integrada de percepción frontal y riesgo contextual. De este modo permite verificar la estructura, orientación, actualización y coherencia de los comandos de intensidad sin depender de la interfaz háptica física; no sustituye su validación perceptual. El nodo `depth_obstacle_filter_node` publica adicionalmente tópicos de depuración: nube de puntos cruda (`/depth_obstacle_filter/debug/depth_cloud`), nube alineada a gravedad (`/depth_obstacle_filter/debug/raw_cloud`), suelo detectado (`/depth_obstacle_filter/debug/ground_cloud`), puntos descartados por encima de la cámara (`/depth_obstacle_filter/debug/ceiling_cloud`) y altura estimada de la cámara sobre el suelo (`/depth_obstacle_filter/camera_height`), todos visualizables con RViz2. El detalle exhaustivo de tópicos ROS 2, incluyendo productores, consumidores, tipos de mensaje y correspondencia con el código fuente, se presenta en el Anexo [11](#).

Capítulo 5

Diseño del método (pipeline de percepción)

Capítulo 6

Implementación en ROS 2

Capítulo 7

Metodología experimental

Capítulo 8

Resultados

Capítulo 9

Discusión

Capítulo 10

Conclusiones

Capítulo 11

Trabajo futuro

Anexos

Anexo A: Trazabilidad de nodos y código fuente

A continuación se exponen tres tablas de referencia que relacionan los nodos principales del sistema, sus paquetes y los tópicos ROS 2 relevantes de entrada y salida. Esta trazabilidad facilita identificar, para cada bloque funcional descrito en los capítulos de Arquitectura, Método e Implementación, qué información consume y qué información publica. Las Tablas [1](#), [2](#) y [3](#) corresponden a la configuración v2 completa del sistema, esto es, *pipeline filtered*, odometría `rtabmap_odom` y activación de `spatial_awareness`.

Nodo	Paquete	In	Out
camera (realsense2_camera_node)	realsense2_camera	hardware	/camera/camera/imu, /camera/camera/color/image_raw, /camera/camera/color/camera_info, /camera/camera/aligned_depth_to_color/image_raw, /camera/camera/depth/image_rect_raw, /camera/camera/depth/camera_info
imu_filter_madgwick	imu_filter_madgwick	/camera/camera/imu	/imu/data_filtered
rgbd_odometry	rtabmap_odom	/camera/camera/color/image_raw, /camera/camera/color/camera_info, /camera/camera/aligned_depth_to_color/image_raw, /imu/data_filtered	nav_odom
depth_obstacle_filter	depth_obstacle_filter	/camera/camera/depth/image_rect_raw, /camera/camera/depth/camera_info, /camera/camera/accel/sample	/depth_obstacle_filter/obstacle_cloud, /depth_obstacle_filter/free_endpoints, /depth_obstacle_filter/camera_height, /depth_obstacle_filter/debug/*

Tabla 1: Trazabilidad de adquisición, prefiltrado IMU y percepción geométrica.

Nodo	Paquete	In	Out
local_mapper	local_mapper	/depth_obstacle_filter/obstacle_cloud, /depth_obstacle_filter/free_endpoints, nav_odom, /depth_obstacle_filter/camera_height	/local_mapper/occupancy_grid, TF odom → gravity_aligned_frame
obstacle_grid_encoder	depth_grid_encoder	/depth_obstacle_filter/obstacle_cloud, /depth_obstacle_filter/camera_height	/perception/depth_grid, /perception/depth_grid/aperture_state_deg
spatial_awareness	spatial_awareness	/local_mapper/occupancy_grid, nav_odom, /perception/depth_grid/aperture_state_deg	/spatial_awareness/collision_risk, /spatial_awareness/debug/markers

Tabla 2: Trazabilidad de mapeo local, codificación de grilla y conciencia espacial.

Nodo	Paquete	In	Out
haptic_grid_generator	haptic_grid_generator	/perception/depth_grid, awareness/collision_risk	/spatial_awareness /perception/haptic_grid
grid_to_motor	hardware_manager	/perception/haptic_grid	Trama UART
depth_grid_heatmap	output_viewer	/perception/haptic_grid, awareness/collision_risk	/spatial_awareness visualización
nav_bringup (launch)	nav_bringup	argumentos de ejecución y configuración general	orquestración del sistema completo

Tabla 3: Trazabilidad de fusión háptica, salida, visualización y orquestación.

Bibliografía

- [1] S. D. Novich y D. M. Eagleman, «Using space and time to encode vibrotactile information: toward an estimate of the skin's achievable throughput,» *Experimental Brain Research*, vol. 233, n.º 10, págs. 2777-2788, 2015. DOI: [10.1007/s00221-015-4346-1](https://doi.org/10.1007/s00221-015-4346-1). dirección: https://eagleman.com/papers/novich_eagleman_2015.pdf.
- [2] V. Santos, R. B. Medeiros, M. T. Rocha y col., «Wearable Haptic Device as Mobility Aid for Blind People: Electronic Cane - Wearable Device for Mobility of Blind People,» *ResearchGate*, 2023. dirección: https://www.researchgate.net/profile/Vagner-Dos-Santos-3/publication/367149471_Wearable_Haptic_Device_as_Mobility_Aid_for_Blind_People_Electronic_Cane_-_Wearable_Device_for_Mobility_of_Blind_People/links/63ceccf66fe15d6a57426662/Wearable-Haptic-Device-as-Mobility-Aid-for-Blind-People-Electronic-Cane-Wearable-Device-for-Mobility-of-Blind-People.pdf.
- [3] A. L. Petsiuk y J. M. Pearce, «Low-Cost Open Source Ultrasound-Sensing Based Navigational Support for the Visually Impaired,» *Sensors*, vol. 19, n.º 17, pág. 3783, 2019. DOI: [10.3390/s19173783](https://doi.org/10.3390/s19173783). dirección: <https://www.mdpi.com/1424-8220/19/17/3783>.
- [4] I. RealSense™, *D400 Series Product Family - Datasheet*, [download-datasheet](#), 2019.
- [5] P. Bach-y-Rita, C. C. Collins, F. A. Saunders, B. White y L. Scadden, «Vision Substitution by Tactile Image Projection,» *Nature*, vol. 221, n.º 5184, págs. 963-964, 1969. DOI: [10.1038/221963a0](https://doi.org/10.1038/221963a0).
- [6] Y. H. Lee y G. Medioni, «RGB-D Camera Based Wearable Navigation System for the Visually Impaired,» *Computer Vision and Image Understanding*, vol. 149, págs. 3-20, 2016. DOI: [10.1016/j.cviu.2016.03.019](https://doi.org/10.1016/j.cviu.2016.03.019).
- [7] H. Zhang, L. Jin y C. Ye, «An RGB-D Camera Based Visual Positioning System for Assistive Navigation by a Robotic Navigation Aid,» *IEEE/CAA Journal of Automatica Sinica*, vol. 8, n.º 8, págs. 1389-1400, 2021. DOI: [10.1109/JAS.2021.1004084](https://doi.org/10.1109/JAS.2021.1004084).
- [8] Seventh Sense OÜ, *SuperBrain 1: Advanced Assistive Technology for the Blind*, <https://7sense.ee/superbrain-1/>, Accessed: 2026, 2024.
- [9] R. P. Foundation, *Raspberry Pi Documentation*, Accessed: 2024-10-04, 2012. dirección: <https://www.raspberrypi.org/documentation/>.
- [10] Open Source Robotics Foundation, *ROS: Robot Operating System*, <https://www.ros.org/>, Accessed: 2024-10-04, 2007.
- [11] A. Elfes, «Using Occupancy Grids for Mobile Robot Perception and Navigation,» *Computer*, vol. 22, n.º 6, págs. 46-57, 1989. DOI: [10.1109/2.30720](https://doi.org/10.1109/2.30720).
- [12] S. Thrun, W. Burgard y D. Fox, *Probabilistic Robotics*. MIT Press, 2005, ISBN: 978-0-262-20162-9.

- [13] D. Dakopoulos y N. G. Bourbakis, «Wearable Obstacle Avoidance Electronic Travel Aids for Blind: A Survey,» *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, vol. 40, n.º 1, págs. 25-35, 2010. DOI: [10.1109/TSMCC.2009.2021255](https://doi.org/10.1109/TSMCC.2009.2021255).
- [14] K. Yang, K. Wang, W. Hu y J. Bai, «Expanding the Detection of Traversable Area with RealSense for the Visually Impaired,» *Sensors*, vol. 16, n.º 11, pág. 1954, 2016. DOI: [10.3390/s16111954](https://doi.org/10.3390/s16111954).
- [15] O. Brock y O. Khatib, «High-Speed Navigation Using the Global Dynamic Window Approach,» págs. 341-346, 1999. DOI: [10.1109/ROBOT.1999.770002](https://doi.org/10.1109/ROBOT.1999.770002). dirección: https://khatib.stanford.edu/publications/pdfs/Brock_1999_ICRA.pdf.
- [16] J. E. Bresenham, «Algorithm for Computer Control of a Digital Plotter,» *IBM Systems Journal*, vol. 4, n.º 1, págs. 25-30, 1965. DOI: [10.1147/sj.41.0025](https://doi.org/10.1147/sj.41.0025).
- [17] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss y W. Burgard, «OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,» en *Autonomous Robots*, vol. 34, 2013, págs. 189-206. DOI: [10.1007/s10514-012-9321-0](https://doi.org/10.1007/s10514-012-9321-0). dirección: <https://octomap.github.io>.
- [18] S. Macenski, F. Martin, R. White y J. G. Clavero, «The Marathon 2: A Navigation System,» *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, págs. 2718-2725, 2020. DOI: [10.1109/IROS45743.2020.9341207](https://doi.org/10.1109/IROS45743.2020.9341207).
- [19] O. J. Woodman, «An introduction to inertial navigation,» University of Cambridge, Computer Laboratory, inf. téc. UCAM-CL-TR-696, 2007. dirección: <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-696.pdf>.
- [20] C. Forster, L. Carlone, F. Dellaert y D. Scaramuzza, «On-Manifold Preintegration for Real-Time Visual-Inertial Odometry,» *IEEE Transactions on Robotics*, vol. 33, n.º 1, págs. 1-21, 2017. DOI: [10.1109/TR0.2016.2597321](https://doi.org/10.1109/TR0.2016.2597321).
- [21] A. I. Mourikis y S. I. Roumeliotis, «A Multi-State Constraint Kalman Filter for Vision-aided Inertial Navigation,» en *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2007, págs. 3565-3572. DOI: [10.1109/ROBOT.2007.364024](https://doi.org/10.1109/ROBOT.2007.364024).
- [22] T. Qin, P. Li y S. Shen, «VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,» *IEEE Transactions on Robotics*, vol. 34, n.º 4, págs. 1004-1020, 2018. DOI: [10.1109/TR0.2018.2853729](https://doi.org/10.1109/TR0.2018.2853729).
- [23] M. Labbé y F. Michaud, «RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation,» *Journal of Field Robotics*, vol. 36, n.º 2, págs. 416-446, 2019. DOI: [10.1002/rob.21831](https://doi.org/10.1002/rob.21831).
- [24] S. O. H. Madgwick, A. J. L. Harrison y R. Vaidyanathan, «Estimation of IMU and MARG orientation using a gradient descent algorithm,» *2011 IEEE International Conference on Rehabilitation Robotics*, págs. 1-7, 2011. DOI: [10.1109/ICORR.2011.5975346](https://doi.org/10.1109/ICORR.2011.5975346).
- [25] Intel Corporation, *Tuning Intel RealSense Depth Cameras for Best Performance*, <https://dev.realsenseai.com/docs/tuning-depth-cameras-for-best-performance>, Accessed: 2026, 2020.
- [26] R. Hartley y A. Zisserman, *Multiple View Geometry in Computer Vision*, 2.^a ed. Cambridge University Press, 2004, ISBN: 978-0-521-54051-3.

- [27] D. Scharstein y R. Szeliski, «A Taxonomy and Evaluation of Dense Two-Frame Stereo Correspondence Algorithms,» *International Journal of Computer Vision*, vol. 47, n.º 1-3, págs. 7-42, 2002. DOI: [10.1023/A:1014573219977](https://doi.org/10.1023/A:1014573219977). dirección: <https://link.springer.com/article/10.1023/A:1014573219977>.
- [28] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4th. Pearson, 2018, ISBN: 978-0133356724.
- [29] SBS Implementers Forum, «System Management Bus (SMBus) Specification, Version 2.0,» SBS Implementers Forum, inf. téc., 2000, Sección 4.1.1: Packet Error Code (CRC-8). dirección: <http://smbus.org/specs/smbus20.pdf>.
- [30] H. B. Menz, S. R. Lord y R. C. Fitzpatrick, «Acceleration patterns of the head and pelvis when walking on level and irregular surfaces,» *Gait & Posture*, vol. 18, n.º 1, págs. 35-46, 2003. DOI: [10.1016/s0966-6362\(02\)00159-5](https://doi.org/10.1016/s0966-6362(02)00159-5).
- [31] A. V. Oppenheim y R. W. Schaffer, *Discrete-Time Signal Processing*, 2.^a ed. Prentice Hall, 1999, ISBN: 0-13-754920-2.
- [32] S. O. H. Madgwick, «An Efficient Orientation Filter for Inertial and Inertial/Magnetic Sensor Arrays,» Internal report, 30 de abr. de 2010. dirección: https://x-io.co.uk/downloads/madgwick_internal_report.pdf (visitado 05-08-2026).
- [33] ROS Enhancement Proposals, *REP-103: Standard Units of Measure and Coordinate Conventions*, <https://www.ros.org/reps/rep-0103.html>, Accessed: 2026, 2010.